POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Software Engineering 2 Requirements Analysis and Specification Document

Author(s): **Bellosi Jacopo - 10806471**
Montanelli Matteo - 10829541
Tempobono Manuel - 10789942

Academic Year: 2025-2026
Version: 1.0
Release date: 23/12/2025

Contents

1 | Introduction

The Best Bike Paths (BBP) initiative is architectured with the intent to facilitate Cyclists in both enhancing their riding experiences and contributing verifiable data regarding bike paths. Cyclists generate path reports and obstacle notifications either manually or by verifying sensor-generated suggestions, and these reports are systematically organized to assist in efficient path exploration. The objective of the project is to deliver a robust and user-centric platform enabling Cyclists and Visitors to identify optimized paths, share substantive empirical evidence, and maintain the currency of path quality. This document, herein referred to as the Requirements Analysis and Specification Document (RASD), encompasses comprehensive information requisite for the seamless implementation of the BBP system, with a particular emphasis on the interactions between the system and the diverse actors interfacing with it.

1.1. Purpose

This document provides a detailed description of the BBP system for the developers who must implement the requirements. It is addressed to the developers and could be used as an agreement between the customer and the contractors. The document is also intended to provide the customer with a clear and unambiguous description of the system's functionalities and constraints, allowing the customer to validate the requirements and to verify if the system meets the expectations.

1.1.1. Goals

Below there's a table that lists all the goals of the BBP system:

ID	Description
G1	Support personal cycling activity tracking
G2	To gather, aggregate, and provide users with actionable data regarding cycle path conditions (e.g., surface status, obstacles, potholes) to support path planning and inform maintenance entities.
G3	Facilitating the discovery and navigation of suitable paths
G4	To enhance trip analysis by enriching recorded user trips with relevant meteorological and physical data retrieved from external services.
G5	To establish a community-based path evaluation system by (a) enabling registered users to submit publishable path information, and (b) giving the users the most reliable information status possible.

Table 1.1: Goals table.

1.1.2. Scope

The scope of the Best Bike Paths (BBP) system is to define a software platform, consisting of a mobile application and a backend system, designed to assist cyclists through intelligent path discovery, personal trip recording, and collaborative path evaluation. The system's primary scope is to enable both Anonymous and Registered Users to find, assess, and share cycling paths based on real, user-generated data and sensor-based observations.

The functionalities within the scope of the system include several key features. Users can track their trips either manually or automatically using the sensors on their smartphones. It also calculates scores for each path based on feedback from the community. These scores are determined by considering both freshness of the data and the overall consensus among users. In addition, external weather data are integrated to make the information even more accurate and useful. Although anonymous users can explore public data, only registered users can actively contribute—such as adding new data, confirming anomalies, and customizing their path recommendations.

1.1.3. World Phenomena

ID	Description
WP1	A User wants to create a new account or manage their existing account (e.g., login, logout, settings)
WP2	A User wants to find a cycling path between points A and B, and view the details of the selected path (e.g., status, weather, elevation gain).
WP3	A User wants to record a Trip by navigating on an existing path, and (if registered) save the completed trip in their personal archive
WP4	A Registered User wants to record a new path by cycling (automatic mode), or plan a new path "at the desk" by drawing it on the map (manual mode).
WP5	When the user wants to navigate a path the BBP invoke the external navigation service passing the travel data.
WP6	A Registered User wants to manually report a problem (e.g., pothole, obstacle) on a path.
WP7	A Registered User wants to manually update the status on a path.
WP8	A Registered User wants to confirm/reject a problem report (e.g., pothole) automatically detected by the app.
WP9	A Registered User wants to view their trips.

Table 1.2: World Phenomenas.

1.1.4. Shared phenomena

ID	Description	Controller	Observer
SP1	The User enters textual data (credentials, origin, destination).	User	BBP
SP2	The User interacts with the interface by pressing buttons (e.g., Login, Save, Publish, Search).	User	BBP
SP3	The User starts automatic trip recording.	User	BBP
SP4	The User stops automatic trip recording.	User	BBP
SP5	The User report the status of a path (e.g., Optimal, Requires maintenance).	User	BBP
SP6	The User selects a specific path from the displayed result list.	User	BBP
SP7	BBP displays informational or error messages (e.g., Save completed, GPS error).	BBP	User
SP8	BBP displays the interactive map.	BBP	User
SP9	BBP renders the recorded path trace on the map.	BBP	User
SP10	BBP shows real-time statistics (speed, distance, duration) during recording.	BBP	User
SP11	BBP displays the trip summary statistics (average speed, elevation gain, total distance).	BBP	User
SP12	BBP shows a ranked list of paths based on the computed score.	BBP	User
SP13	BBP displays the aggregated (merged) status of a path.	BBP	User
SP14	BBP shows weather data associated with the path (temperature, wind, conditions).	BBP	User
SP15	BBP displays a confirmation request for an automatic detection (popup / notification).	BBP	User
SP16	The GPS sensor provides coordinates (latitude, longitude) to BBP.	GPS Sensor	BBP
SP17	The accelerometer sends acceleration vector measurements (x, y, z) to BBP.	Accelerometer	BBP
SP18	The gyroscope provides rotation data (x, y, z) to BBP.	Gyroscope	BBP
SP19	BBP sends an API request (e.g., with path coordinates/timestamps) to the external weather service.	BBP	Weather API
SP20	The external weather service sends JSON data on temperature, wind, and atmospheric conditions.	Weather API	BBP
SP21	The User starts a navigation.	User	BBP

SP22	After SP21, BBP sends a request (e.g., URL Scheme / Intent) to the device's Operating System to open an external navigation app with the selected path/destination.	BBP	OS / External Navigator
SP23	The User exit the external navigation app, and terminate the navigation on BBP.	User	BBP

Table 1.3: Shared Phenomenas.

1.2. Definition, Acronyms, Abbreviations

Acronyms	**Definition**
RASD	Requirements Analysis & Specification Document
RCY	Registred Cyclist
ACY	Anonymous Cyclist
BBP	BestBikePaths
User	All RCYs and ACYs
API	Application Programming Interface
DAX	Domain Assumption X
SPX	Shared Phenomena X
WPX	World Phenomena X
RX	Requirement X

Table 1.4: Acronyms used in the document.

1.3. Revision history

- **Version 1.0** - 12/11/2025

1.4. Reference Documents

- Specification Document Assignment
- IEEE Standard Documentation For RASD

1.5. Document Structure

The document is organized into six main sections, each focusing on a specific aspect, as summarized below.

Introduction: The first section outlines the project's goals and purpose, and provides a brief overview of the global and shared phenomena. It also includes key abbreviations and definitions necessary to understand the problem.

Overall Description: The second section offers a general overview of the problem, explaining the domain, main scenarios, product and user characteristics, as well as the related assumptions, dependencies, and constraints.

Specific Requirements: The third section presents a detailed analysis of the system requirements, covering external interfaces, functional aspects, and performance expectations.

Formal Analysis Using Alloy: The fourth section uses Alloy to perform a formal validation of the model introduced earlier. It reports the results of the executed checks and the verified assertions.

Effort Spent: The fifth section details the contribution of each group member to the development of this document.

References: The last section lists all bibliographic sources and materials used during the document's preparation.

2 | Overall Description

2.1. Product perspesctive

2.1.1. Scenarios

Authentication and Access Scenarios

Scenario ID	**1.1– Registration (Success)**
Actor	Anna (new user)
Preconditions	Internet connection available (DA12).
Description	Anna opens the BBP app and taps "Register." She enters her email and password. The system creates the account, authenticates her, and redirects her to the main map screen as a registered user.
Postconditions	The user account is created, and Anna is logged in.

Scenario ID	**1.2– Login (Success)**
Actor	Carlo (registered user)
Preconditions	Valid credentials and Internet connection (DA13).
Description	Carlo opens the app, selects "Login," enters his credentials ("CarloMTB," password), and presses "Sign In." The BBP system verifies them and grants access, showing the main map screen.
Postconditions	Carlo is authenticated and redirected to the main map screen.

Scenario ID	1.3– Login (Exception – Wrong Password)
Actor	Carlo (registered user)
Preconditions	Incorrect password entered.
Description	Carlo tries to log in but mistypes his password. The BBP system verifies the credentials and, upon failure, shows an error message: "Invalid username or password," keeping him on the login page.
Postconditions	Login fails; no session is created.

Path Consultation Scenarios

Scenario ID	2.1– Search and Display (Success)
Actor	Marco (anonymous user)
Preconditions	Internet connection available (DA13).
Description	Marco uses the search bar to set "Stazione Garibaldi" as the origin and "Parco di Monza" as the destination. The BBP system processes the query, computes Path scores, and shows an ordered list. Marco selects "Path A," and the app displays the Path, details (distance, elevation), and weather forecast (DA12).
Postconditions	Paths are displayed; user views the selected Path and related data.

Scenario ID	2.2– System Logic (Scoring & Merging) – Key Scenario
Actor	BBP System
Preconditions	User search triggered (Scenario 2.1). Data from past Path reports available (DA8)(DA9).
Description	"Path B" has three status updates: two "Requires Maintenance" and one "Optimal." The system's merging algorithm aggregates the data based on freshness (DA8 and majority (DA9), assigning "Requires Maintenance." The scoring algorithm uses this to compute a final score of 6.5.
Postconditions	Aggregated Path status and updated score are displayed to the user.

Scenario ID	2.3– Search (Exception – No Path Found)
Actor	Marco (user)
Preconditions	Query between remote or unsupported locations.
Description	Marco searches for a Path, but no publishable results match the criteria. The BBP system displays: "No BBP Path found. Be the first to create one!"
Postconditions	No results displayed; user invited to create a Path.

Scenario ID	2.4– Navigation
Actor	Marco (user)
Preconditions	A Path has been selected. Accurate GPS assumed (DA2).
Description	Marco presses "Follow Path", The application imports the Path into an external navigation app, which then directs him along the path. Eventually, Marco returns to BBP, stops the trip recording, and BBP provides him with the trip data and suggests registering Marco so he can save his trip details.
Postconditions	Trip completed; user view summary and may register.

Path Creation Scenarios

Scenario ID	3.1– Manual Path Creation (Success)
Actor	Anna (registered user)
Preconditions	User logged in; understands options (DA10).
Description	From her profile, Anna selects "Create Manual Path." She draws the Path on the map, names it "Tuesday Ride," activates "Make Public," and presses "Save" (DA13). The Path becomes available to others.
Postconditions	The new Path is created and visible to all users.

Scenario ID	**3.2– Automatic Recording (Success) Key Scenario**
Actor	Carlo (registered user)
Preconditions	Accurate GPS data (DA2), reliable threshold (DA4), device fixed to bike (DA5).
Description	Carlo presses "Start Recording." The app tracks his movement, displaying real-time data. The rides go really smoothly, but later, a strong shock occurs due to a pothole. Sensors send data spikes to BBP (R24). Upon completion, BBP presents the logged `sensor_events` for review (R28); Carlo, acting in good faith (DA7), confirms the correct detections (R29).The app displays the final summary and statistics (R31). During saving, Carlo selects the visibility for both Trip and Path (R39, R26, R25) and the Trip is stored in his archive (R33) (DA13).
Postconditions	Trip data ('Elapsed Time' and confirmed detections) are stored in Carlo's archive. 'Average Speed' is calculated using 'Elapsed Time'.

Scenario ID	**3.3– Automatic Recording (False Detection & User Correction)**
Actor	Carlo (registered user)
Preconditions	Same as Scenario 3.2; false detection occurs.
Description	Carlo's recording continues uninterrupted, but the detected spike results from jumping off a curb. After terminating the recording, BBP presents the logged `sensor_events` for review (R28). Carlo (acting in good faith, (DA7)) presses "No, ignore" for incorrect detections (R29), and then completes the trip as in Scenario 3.2.
Postconditions	False detections are removed; valid trip data saved.

Scenario ID	**3.4– Automatic Recording (No Detections)**
Actor	Carlo (registered user)
Preconditions	Flat and obstacle-free Path; no sensor spikes (DA4).

Description	Carlo rides on a smooth Path. No anomalies are detected (R24).When pressing "End and Save," BBP skips the "Review" step (R28) and directly shows the summary screen.
Postconditions	Trip saved successfully without detected issues.

2.1.2. Class diagrams

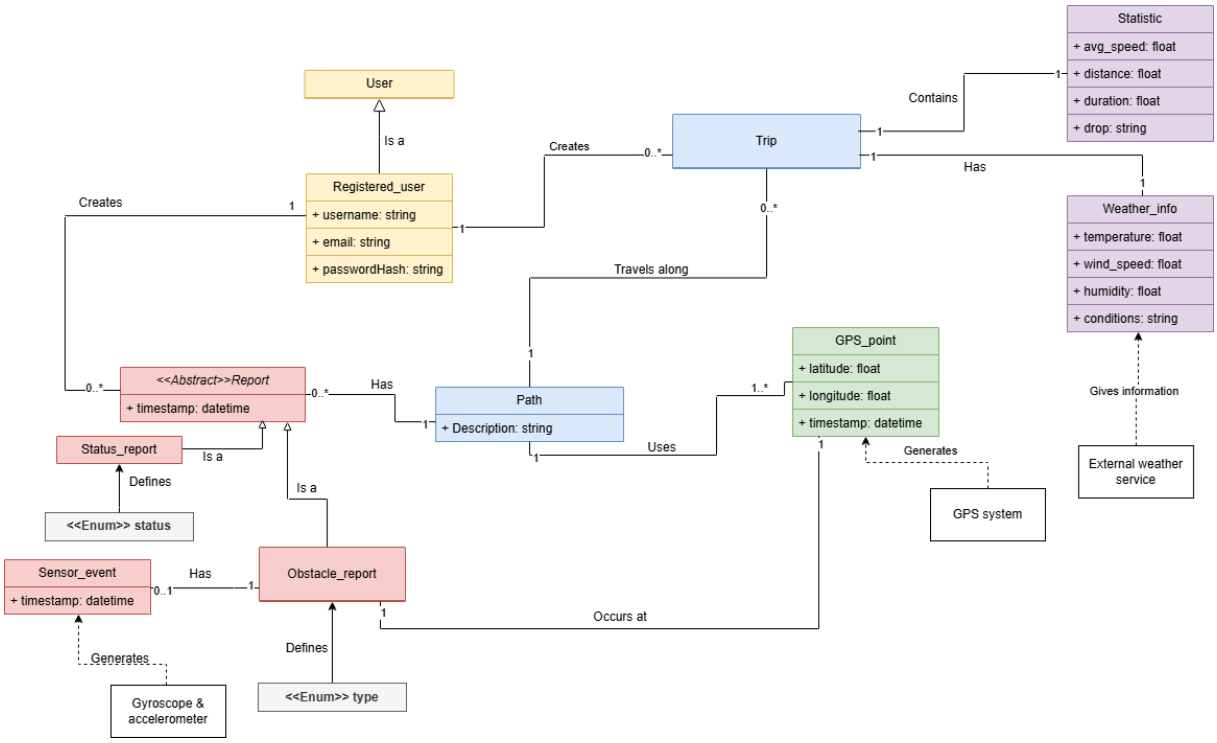

Figure 2.1: High level Class Diagram.

Class Diagram Description

The system's structure revolves around the base **User** entity. The `Registered_user` class extends `User`, incorporating attributes like `username`, `email`, and `passwordHash` necessary for authentication and user identification. Each registered user is associated with their recorded activities and contributions.

A core distinction is made between a `Path` (the "model" of a route) and a `Trip` (a user's "performance" on that route).

The `Path` class represents the route's definition. It is composed of `GPS_points` that define its geometry (R36). A `Path` has a `pathVisibility` (Public/Private), which determines if it is discoverable via search (R7, R25) (visibility chosen during saving, R39).

A `Trip` represents a specific, private cycling session recorded by a `Registered_user`. It is also fundamentally defined by its own sequence of `GPS_points` (the "trace") recorded during the session (R21), as described in the requirements. Furthermore, each `Trip` is linked via composition to exactly one `Statistic` object. This object holds calculated data

like avg_speed, distance, and duration (which represents the total 'Elapsed Time', per R22). The avg_speed is calculated using this total duration ('Elapsed Time') and 'Total Distance'(R31). Each Trip is also associated with one Weather_info object (R32).

Crucially, the diagram models the core business logic: a Path is associated with all the Trips performed on it (a 1-to-N "performances" or "Is connected to" relation). A Trip has its own tripVisibility (Public/Private), which dictates whether its statistics are used in community aggregation and scoring (R9, R10, R26) (visibility chosen during saving, R39).

A Path also serves as an aggregation point for community feedback, being associated with multiple Report instances. The «Abstract» Report class acts as the base for all user-submitted information regarding a Path, containing a vital timestamp attribute used for the merging algorithm's "freshness" logic (R9). It has two concrete specializations:

- Status_report: Captures user assessments of the Path's condition via the «Enum» status attribute (R37).
- Obstacle_report: Represents specific issues like potholes (R38), categorized by an «Enum» type. Importantly, it maintains a [0..1] association with a sensor_event, linking a confirmed report (R30) back to the raw sensor data that potentially triggered its detection (R24).

The sensor_event class models the raw detection triggered by device sensors (accelerometer/gyroscope) during a Trip (R23), storing its location and timestamp before user confirmation (R28). The interaction with external systems (conceptually noted, e.g., External tools) like device sensors and weather services is fundamental for data acquisition.

The diagram effectively delineates a structure that distinguishes **user management** from the fundamental elements: the Path, which acts as the public-facing and searchable model, and the Trip, representing the user's private performance that may optionally share its statistics with the Path. Additionally, it illustrates the organized **reporting system** through the use of Report inheritance and outlines the **automatic detection** process via sensor_event.

2.1.3. State diagrams

Unlike Activity Diagrams, which model process flows, the following State Machine Diagrams model the dynamic lifecycle of specific objects within the domain in response to events. For the BBP system, two objects were identified as having complex, requirement-relevant lifecycles: Trip and sensor_event.

Trip State Diagram. This diagram (Figure 2.2) models the lifecycle of a `Trip` recording session (UC7). The object begins in the `Idle` (or Ready) state. When the `RCY` initiates the recording (R20, `tapsStartRecording()`), the object transitions to the `Recording` state. In this state, it actively records GPS (R21) and sensor (R23) data, and the 'Elapsed Time' (R22) runs continuously.

The object remains in the `Recording` state without interruption until the `RCY` explicitly triggers the `tapsStopRecording()` event (R27). At this point, the object transitions to the `Stopped (Pending Save)` state, where the collected data is passed to the `Save trip and path` (UC9) use case. Depending on the user's final choice, the object transitions to one of two final states: `Saved` (R33) or `Discarded`.

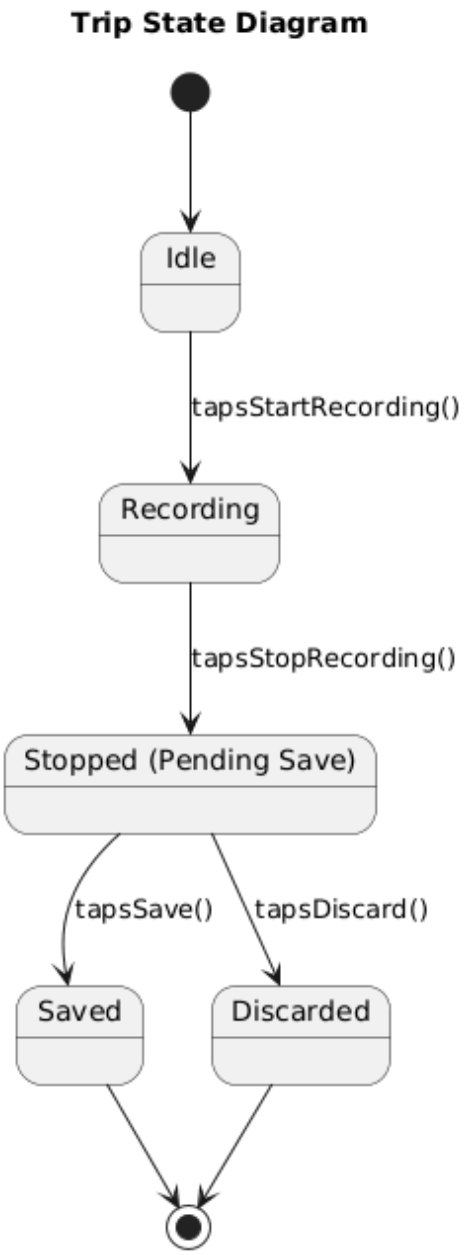

Figure 2.2: State Diagram for the `Trip` lifecycle.

sensor_event State Diagram. This diagram (Figure 2.3) models the lifecycle of a single `sensor_event` object. An event is created and transitions to the `Logged` state the moment the device sensors (R23) exceed the predefined detection threshold (R24).

It remains in this state for the entire duration of the recording (UC7). When the `RCY` stops the recording (R27) and the `Save trip and path` (UC9) summary screen is displayed, the event transitions to the `AwaitingConfirmation` state (R28). At this point, based on the user's decision (R29) during the `Review Detections` (UC8), the event transitions to

one of two final states: `Confirmed` (which triggers the creation of an `Obstacle_report`, R30) or `Ignored`.

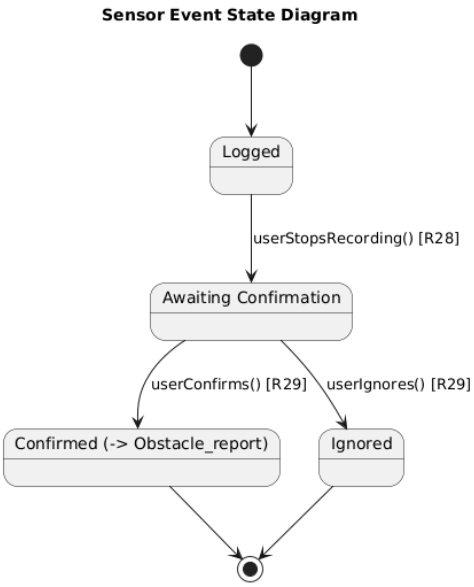

Figure 2.3: State Diagram for the `sensor_event` lifecycle.

2.1.4. Activity diagrams

In this section, the Activity Diagrams of the BBP system are presented representing all the possible activity that a User can perform.

Signup. When a user opts to register for BBP, they are initially presented with the registration option on the interface. Upon tapping the registration button, the system displays the dedicated registration screen. The user then enters the required credentials, such as username, email, and password. Subsequently, BBP performs a check to validate the provided information. If the credentials are deemed valid, the registration is successful, and the user is navigated to the main screen. However, if the validation fails (due to invalid format or existing username/email), the flow loops back, presumably returning the user to the registration screen to correct their input and try again.

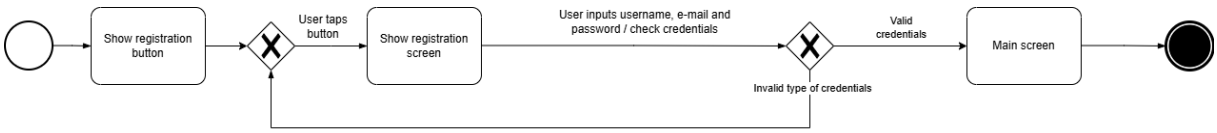

Figure 2.4: Registration activity diagram.

Login. The process begins when the system presents the user with the option to log in, typically via a "Login" button ("Show login button"). If the user chooses to proceed by tapping the button ("User taps button"), the system displays the credentials screen ("Show credentials screen"). Here, the user enters their username and password, after which the system initiates a check of these credentials ("User inputs username and password / checking credentials"). A decision point follows: if the credentials are determined to be valid ("Valid credentials"), the user is successfully authenticated and directed to the main main screen ("Main screen"), concluding the login process. However, if the credentials are found to be incorrect ("Incorrect credentials"), the system displays an error message ("Show error message"). From the error message, the flow loops back to allow the user to re-enter their credentials on the credentials screen, providing another attempt to log in.

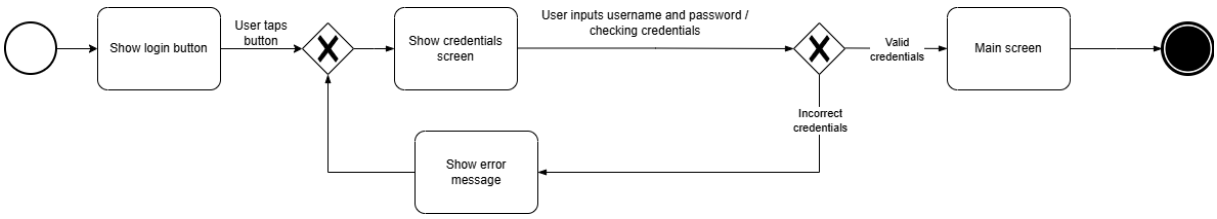

Figure 2.5: Login activity diagram.

Search and Display. This diagram illustrates the user journey for finding bike Paths in BBP. The process begins when the user accesses the search page and inputs their desired start and destination points. The system then enters a processing phase where it identifies relevant Paths and computes their scores, incorporating community-provided status updates through a merging algorithm.

Following this, a check determines if any Paths were found. If no results are available, a message like "Be The First!" is displayed, and the process ends. If results are found, the system presents a list of Paths ordered by score. At this point, the user can either select a specific Path to view its details or abort the search, returning to the initial search page. If a Path is selected, the system displays comprehensive information including the map, calculated statistics, and the aggregated Path status, after which this workflow concludes.

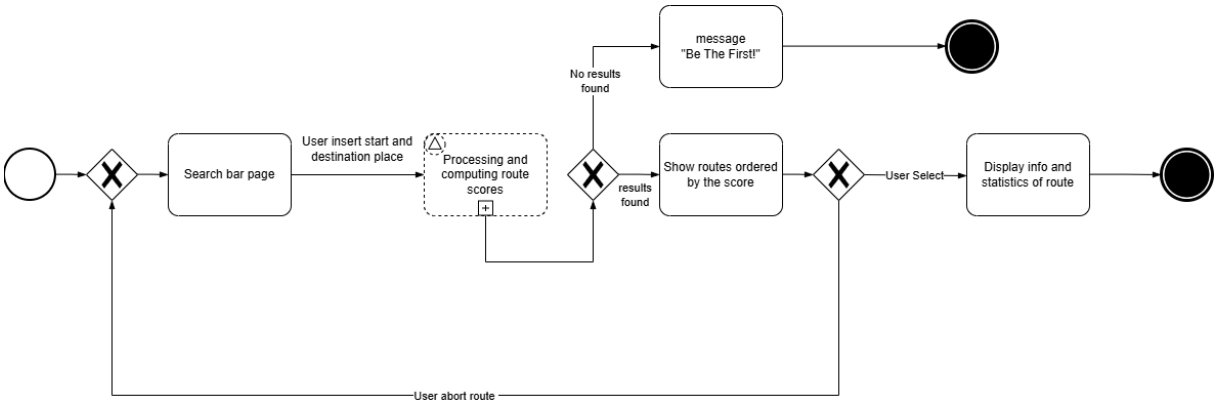

Figure 2.6: Search and Display activity diagram.

Scoring & Merging. It starts when the search triggers the fetching of all past users reports related to the Path.

First, it checks if any reports exist. If there's no data, the process stops for that Path. If reports are found, the system merges them, aggregating user feedback (primarily status reports) to determine a representative condition. A check follows to see if the merging resulted in conflicts (e.g., no clear majority status). If a conflict exists, the freshness rule is applied, prioritizing the most recent reports to decide the final status.

Once a definitive, merged status is established (either directly or after resolving conflicts), the system computes the Path's score . This final score, along with the merged status, is then outputted ("Score computed, Result page") for use in the search results, completing the evaluation workflow.

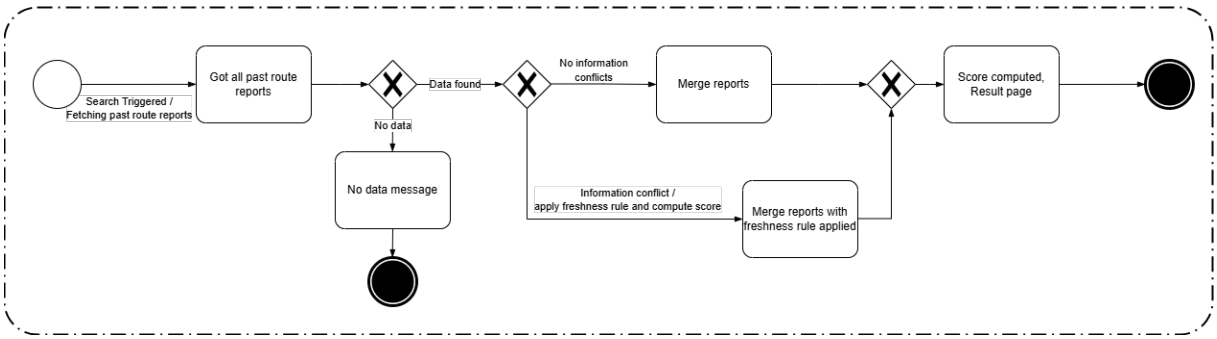

Figure 2.7: Scoring & Merging logic activity diagram.

Navigation. After a Path has been selected from the search results, the User taps "Follow Path". The BBP system then loads the chosen Path onto the External navigation service and begins acquiring GPS data to provide stats data based on the user's current position (GPS assumed to be available per DA2).

When the User reaches the destination, the user go back on BBP that analyses the travel data and checks the user's status. At the end of the trip, the system displays a summary screen. If the user is Anonymous, BBP first shows the trip summary in view-only mode. Then, the system offers registration. If the Anonymous User chooses to register ("User accepted"), BBP guides them through the registration process; upon successful registration, the trip is saved to their new account. If the Anonymous User refuses registration ("User refused"), BBP keeps the summary in view-only mode without permanently storing the ride details, and the process ends.

Conversely, if the User navigating is already Registered, upon arrival at the destination, BBP will show the summary of the trip. BBP then presents an option to view the summary ("Show summary and save button"), the registered user after viewing the summary can choose to save the trip or not.

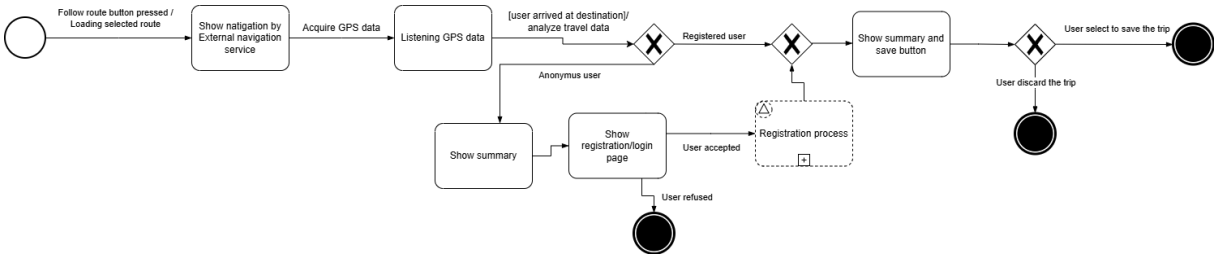

Figure 2.8: Navigation activity diagram.

Manual Path Creation. From their profile or a dedicated menu, the User selects the option to create a new Path manually, initiating the process ("Manual Path Selected"). BBP then presents the Manual Path Editor page, typically displaying a map interface ready for input. The User iteratively builds the Path by interacting with the editor's tools, adding or undoing segments point by point ("User add/undo segment"). This loop continues until the User indicates the Path geometry is complete ("Path Completed? - Yes").

Upon completion, the system may perform an initial calculation of basic geometric properties like distance ("/ calculate stats"). The system then prompts the User, asking if they wish to add specific maintenance or status pins along the Path ("Add Maintenance Pins?"). If the User chooses yes ("Yes"), they utilize the provided tool ("Adding maintenance pin tool") to place these markers ("User add pin"). Once finished adding pins, or if they chose not to add any ("No"), the User proceeds to attach metadata. This involves entering essential details like the Path Name and setting its visibility (Public or Private).

Finally, the User initiates the save action, triggering a system validation check ("saved

checking validity"). If the submitted Path and metadata are deemed invalid ("Invalid")—for example, if a name is missing—the process terminates, likely displaying an error to the User. If the data is valid ("Valid / publish Path"), the system saves the Path. If marked as public, it's indexed and made available to the community; otherwise, it's saved privately to the User's account. The manual creation process then successfully concludes.

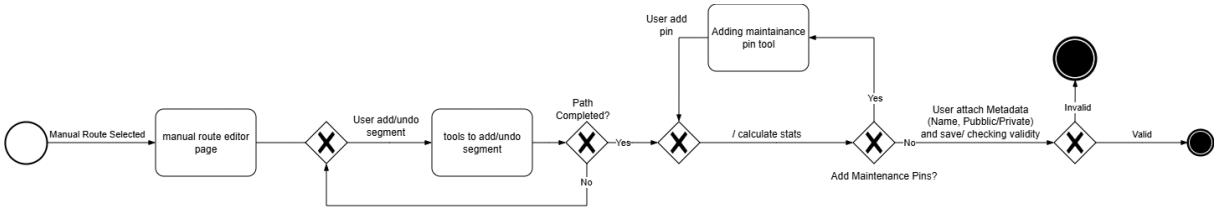

Figure 2.9: Manual Path Creation activity diagram.

Automatic Recording. When the User presses "Start Recording" (R20), BBP initializes the session, updates the display (HUD), and activates the necessary device sensors (R23). Two key processes then run concurrently: the system listens to GPS data to construct the path's geometry (R21) and simultaneously monitors other sensors (accelerometer/gyroscope) to detect potential obstacles (R24).

During the recording session, two key processes run concurrently: the system listens to GPS data to construct the path's geometry (R21) and simultaneously monitors IMU sensors (accelerometer/gyroscope) to detect potential obstacles (R23, R24). BBP continuously logs GPS and sensor data from the moment the user starts the session (R20) until the user explicitly ends it (R27). The session timer shown in the HUD represents the total 'Elapsed Time' between start and stop (R22) and is used for final statistics computation (R31).

Once the User presses "End", both parallel data streams stop. The system first elaborates the recorded path geometry ("Elaborate path") and calculates the final statistics (R31), which will be based on the total Elapsed Time.

The system then performs a crucial check: is a review required? ("Review required? (PendingDetections > 0)") (R28). If a review is necessary ("Yes"), BBP presents an interface showing the locations of the potential detections ("Show where are the detection"), prompting the user to confirm or reject each detection individually (R29, R30).

After the review step is completed (or if no review was needed), the system displays the final trip summary. From this screen, the User can either discard the session and return to the homepage, or save the Trip (R33) and the generated Path (R25). If the User chooses

to save, BBP allows the User to select the visibility options for both Trip and Path (R39) and then shows a confirmation message upon successful saving (R40).

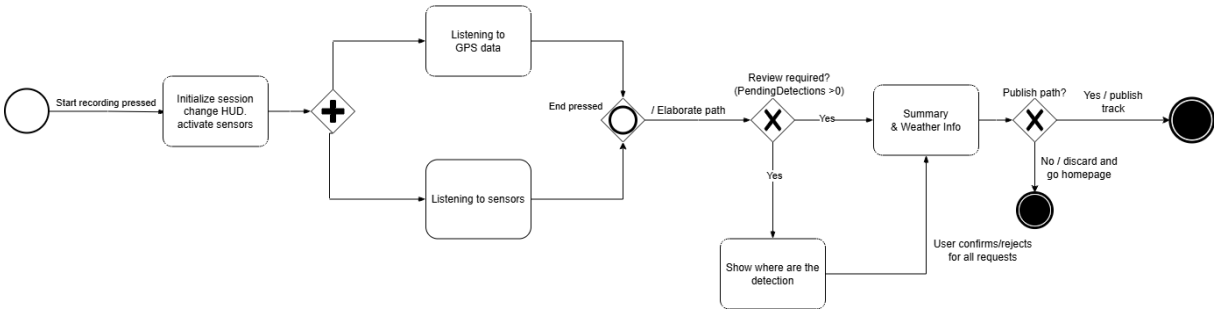

Figure 2.10: Automatic Recording activity diagram.

2.2. Product functions

Here are listed a summary of the main functions of the BBP system:

Signup: The User enters email and password to create a new BBP account; If checks fail (e.g., email already exists), BBP shows an error message, otherwise the user is redirected to the map as a registered user.
Login: The User types credentials (nickname/email and password); if valid, BBP starts the session and shows the main map; otherwise, an error is shown and the User remains on the login page.
Logout: The User ends the current session; subsequent access requires logging in again.
Search Path: The User specifies origin and destination; BBP computes scores and returns an ordered list of publishable Paths.
View Path details: After selecting a Path, BBP displays the polyline, distance, elevation, the merged current status, reported obstacles, and the weather forecast associated with the Path .
Follow Path (Navigation): The User taps "Follow Path"; BBP redirect to an External navigation service and uses GPS to acquire travel data. At the end, BBP show the summary of the trip and offers registration to save the data.
Start Automatic Recording: A registered User starts a session; BBP records the **Trip's geometry** (GPS) and live stats while acquiring motion-sensor data. The system, when the user terminates and decides to terminate the recording, will recreate the total Path.
Automatic Detection Review: At the end of recording, BBP asks the User to con-

firm or reject sensor-detected anomalies (e.g., potholes); only confirmed detections are retained.
End & Save Trip: BBP computes trip statistics and attaches a weather snapshot; the trip (and any confirmed detections) is saved to the User's archive.
Manual Path creation : The User opens the editor, draws the Path, optionally adds maintenance pins or the path status, sets type and visibility, and presses "Save."
Publish/Privacy Path choice: When saving a manually or automatically created Path, the User can make it Public (visible to everyone) or keep it Private in their library.
Publish/Privacy Trip Choice: After a Trip (Navigation or automatic Path creation), the User can make it Public (visible to everyone as statistics of the Path) or keep it Private in their library.
Path Scoring & Merging: For searched Paths, BBP aggregates community reports by majority and freshness to derive the current status and compute the Path score shown in the results.
My Trips & Summaries: Registered Users can view stored trips with summaries (distance, duration, elevation, average speed) and associated weather information.
Notifications & Messages: BBP displays confirmations, success, or error messages during key actions (e.g., save completed, invalid credentials, no Path found).

2.3. User characteristic

There are two categories of BBP users: Anonymous Users and Registered Users. Their main characteristics:

- **Anonymous User (Guest).** Can use BBP without an account to search for Paths between an origin and a destination and visualize them on the map, ranked by score. May follow an existing Path (navigation / view-only trip summary), but cannot save rides or publish information. A network connection and location access are required during use.
- **Registered User.** Creates an account to record and store personal trips, view trip statistics (e.g., total distance, average speed), and—when available—have trips enriched with weather data retrieved from an external service. Can contribute publishable information about bike Paths in two ways: (i) manual mode by entering Path information, and (ii) automated mode by letting BBP acquire GPS plus motion-sensor data (accelerometer/gyroscope) while riding; automated detections are confirmed/corrected by the user before publication. Registered users can also

choose whether contributed information is public (publishable) or kept private.

Both user types benefit from the system's Path scoring and visualization; only registered users can save trips and contribute new information (manual or automated).

2.4. Assumptions, dependencies and constraints

2.4.1. Domain assumptions

ID	Description
DA1	Users must have a mobile device (smartphone) equipped with GPS, accelerometer, and gyroscope sensors.
DA2	The device's GPS sensor provides location data accurate enough to faithfully reconstruct the Path taken.
DA3	Accelerometer and gyroscope data correlate with real-world physical events (e.g., potholes, vibrations).
DA4	A data threshold exists to reliably distinguish significant movements (potholes) from normal biking vibrations.
DA5	The mobile device remains physically attached (e.g., in a pocket or on a mount) to the user or bike during recording.
DA6	The device has sufficient resources (battery, CPU) to sustain an active recording session (GPS + sensors).
DA7	Users act in good faith when providing manual data or confirming automatic detections.
DA8	"Fresh" (recent) information is considered more relevant than older data when describing a Path's status.
DA9	Users provide correct and rational evaluations during Path assessments.
DA10	Users can correctly understand and distinguish the provided status levels (e.g., "Optimal", "Requires Maintenance").
DA11	The external weather service API is available, stable, and provides accurate data.
DA12	The user, while cycling, will not change the track or get lost.
DA13	The device has an active internet connection when searching, saving, or publishing a Path.

DA14	Paths refer to either dedicated bike tracks or roads with low traffic and cycling-compatible speeds (as per the problem statement).
DA15	Users grant permissions for location and motion sensors; otherwise, only manual mode and browsing are available.
DA16	Users will start the recording at the beginning of the trail and end it at the finish line, and are unable to close the app to ensure continuous recording.
DA17	Users will not commit errors while drawing or following the Path

Table 2.12: Domain assumptions.

3 | Specific Requirements

This section details the interfaces between the Best Bike Paths (BBP) system and external entities, including users, hardware components, other software systems, and communication networks. These interfaces define how BBP interacts with its operational environment.

3.1. External interface requirements

3.1.1. User interfaces

The primary user interface for the BBP system will be a native mobile application designed for smartphones (IOS and Android platforms). The interface must prioritize clarity, usability, and robustness, particularly for outdoor use under varying light conditions. It will adopt a map-centric design, prominently displaying geographical data and path information. Key interaction elements will include a clear search bar for origin/destination input, interactive map elements for manual path creation and viewing details, easily accessible buttons for critical functions like starting/stopping automatic recording, and a dedicated Heads-Up Display (HUD) during active recording sessions showing real-time statistics (speed, distance, time) with high legibility. The interface will also incorporate clear feedback mechanisms and prompts for user input, such as review and confirmation of automatically detected sensor events presented at the conclusion of a trip . Navigation between different sections (e.g., Map, Search Results, My Trips, User Profile) must be intuitive and consistent throughout the application.

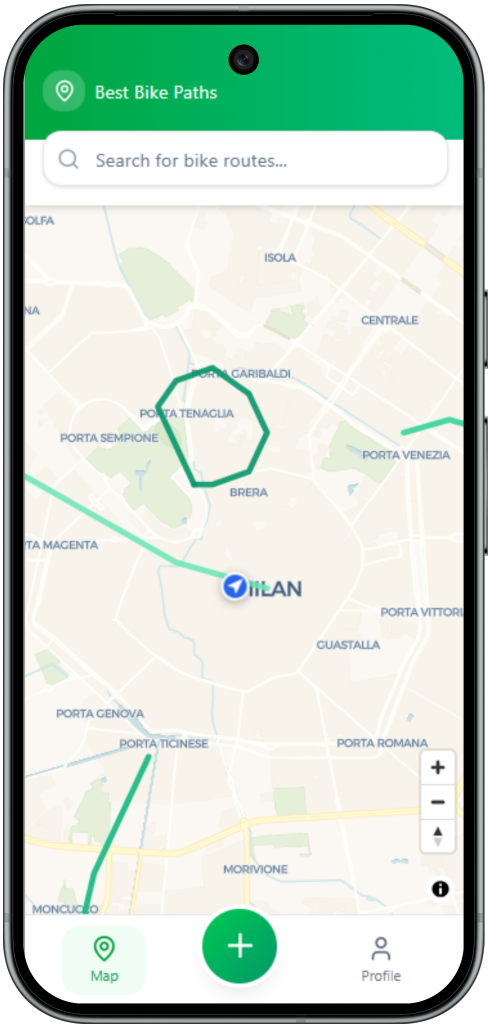

Figure 3.1: View home

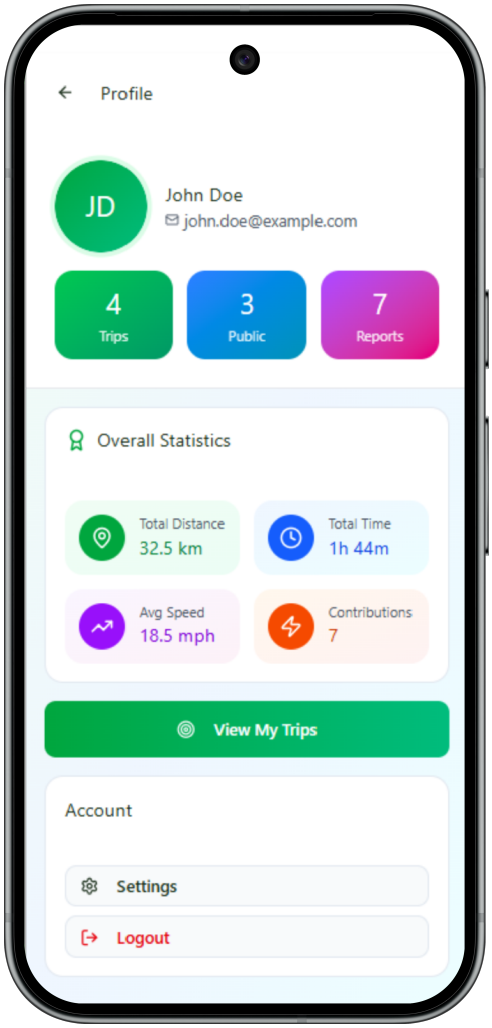

Figure 3.2: View profile

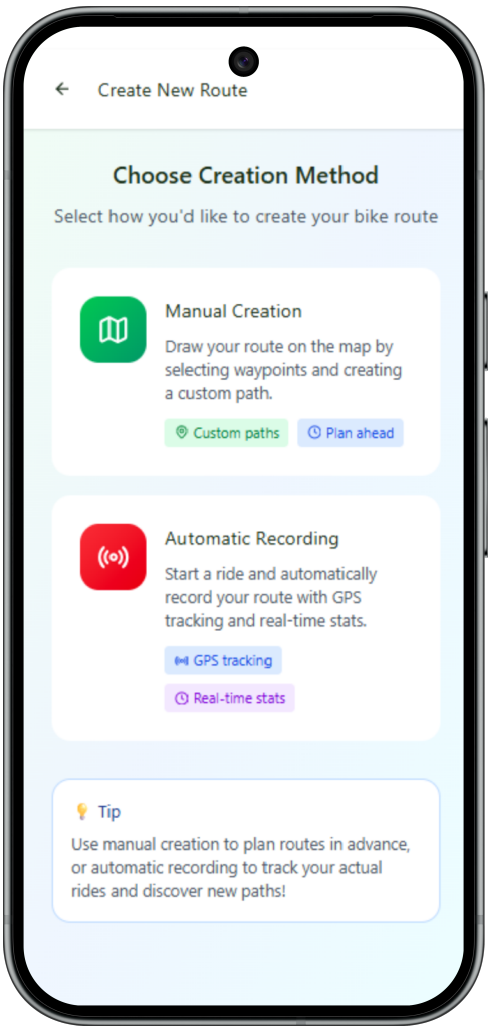

Figure 3.3: View creation mode

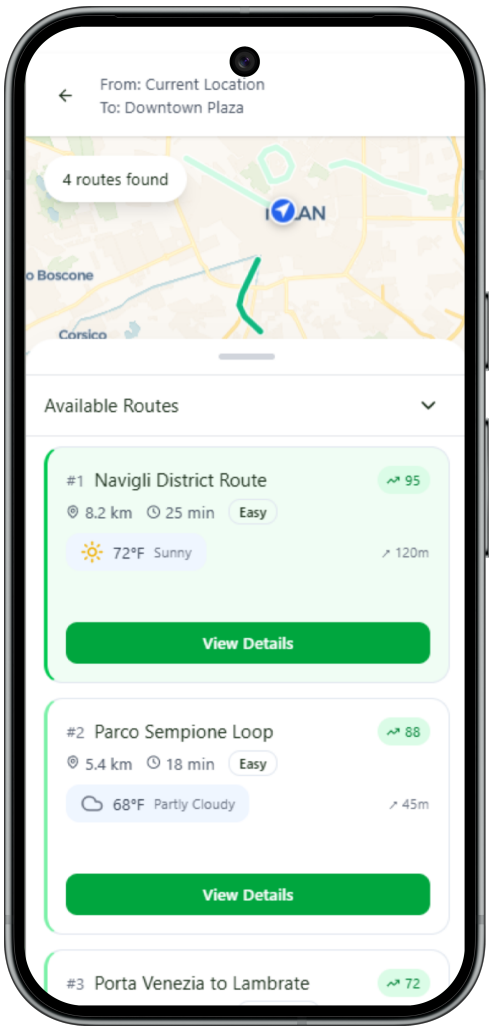

Figure 3.4: View search trip

3.1.2. Hardware interfaces

The BBP system requires direct interaction with specific hardware components integral to the user's mobile device. Foremost is the Global Positioning System (GPS) receiver, from which the application must continuously acquire accurate location data (latitude, longitude, timestamp) to track the user's position during navigation and automatic trip recording. Secondly, BBP must interface with the device's inertial measurement unit (IMU) sensors, specifically the accelerometer and gyroscope. Access to the raw data streams from these sensors is mandatory for the implementation of the automatic obstacle detection feature, enabling BBP to identify significant motion events that may correspond to real-world path anomalies, such as potholes. Furthermore, the application relies on standard smartphone hardware, including the touchscreen display for all user input and output, and potentially the device's vibration motor for non-visual notifications

(e.g., indicating a potential obstacle has been logged). BBP assumes these hardware components are functional and accessible via the device's operating system APIs.

3.1.3. Software interfaces

BBP will interact with several other software entities. A critical interface exists with the mobile device's Operating System (OS). Through OS APIs, BBP must request and manage permissions for accessing hardware sensors (GPS, accelerometer, gyroscope), handle background execution for continuous trip recording, potentially manage local storage for application data, and interact with network services. Another vital software interface is with the External Weather Service. BBP must communicate with this external service via its provided Application Programming Interface (API), likely transmitting location coordinates (and potentially time information) and receiving structured weather forecast data (e.g., in JSON format) containing temperature, wind speed, precipitation probability, and general conditions . The specific protocol and data format will be dictated by the chosen weather service provider.

3.1.4. Communication interfaces

The BBP system inherently requires network connectivity for many of its core functions. An active internet connection, typically via cellular data (e.g., 4G, 5G) or Wi-Fi, is necessary for user authentication (signup/login), searching for public paths, retrieving path details (including merged status and reports) , fetching real-time weather data from the external service , and saving or publishing user-generated trips and reports. Communication with the BBP backend server (which manages user accounts, paths, reports, etc.) will occur over standard secure web protocols(HTTPS), to ensure data integrity and confidentiality during transit. BBP must gracefully handle temporary losses of connectivity where possible, although core online features will be unavailable without an active connection (as per DA12).

3.2. Functional requirements

3.2.1. Requirements

This section lists the specific functional requirements that the Best Bike Paths (BBP) system must fulfill.

ID	Description
R1	BBP allows Users to sign up
R2	BBP shall verify that the username and email provided during registration are not already in use.
R3	BBP shall allow RCY to log in using their username/email and password.
R4	BBP shall provide feedback to the user in case of failed login attempts (e.g., incorrect credentials).
R5	BBP shall allow an RCY to view the personal page with private information.
R6	BBP shall allow a logged-in user to log out.
R7	BBP shall allow users to search for public bike paths by specifying an origin and a destination.
R8	BBP shall retrieve all relevant `Report` instances associated with the paths found during a search.
R9	BBP shall calculate a merged status for each searched path based on its associated `Status_report` instances, applying freshness and majority rules.
R10	BBP shall calculate a score for each searched path based on its reports/conditions and metrics.
R11	BBP shall display search results as a list of paths ordered by the calculated score.
R12	BBP shall allow any user to select a path from the search results to view its details.
R13	BBP shall display the details of a selected path, including its geometry (map), distance, elevation, calculated merged status, and reported obstacles.
R14	BBP shall retrieve and display current weather forecast information for the selected path's location.
R15	BBP shall allow any user to initiate a navigation mode ("Follow Path") for a selected path.
R16	Upon completion of navigation by a user, BBP shall show the summary of the trip (example: avg. speed, distance,...).

Continued on next page

ID	Description
R17	Upon completion of navigation by an anonymous user, BBP shall offer the option to register.
R18	If an anonymous user refuses registration after navigation, BBP shall discard the navigation session data.
R19	If an anonymous user accepts registration after navigation, BBP shall allow them to complete registration and then save the navigated session as a `Trip`.
R20	BBP shall allow an RCY to start an automatic path recording session.
R21	During automatic recording, BBP shall record the user's GPS coordinates sequence.
R22	During automatic recording, BBP shall display real-time statistics (speed, distance, time).
R23	During automatic recording, a system shall monitor accelerometer and gyroscope data.
R24	BBP shall internally log a `sensor_event` when sensor data exceeds a predefined threshold.
R25	BBP shall store the visibility setting of a saved Path as specified by the Registered User, ensuring that Public paths are retrievable via the search function (R7), while Private paths remain strictly accessible only within the creator's personal archive.
R26	BBP shall store the visibility setting of a saved Trip as specified by the Registered User, ensuring that data from Public trips are included in the aggregated Path statistics and scoring algorithms (R9, R10), while data from Private trips are excluded from all community calculations.
R27	BBP shall allow an RCY to stop the automatic trip recording.
R28	Upon stopping recording, if `sensor_event`s were logged, BBP shall present them to the user for review.
R29	BBP shall allow the user to confirm or ignore each `sensor_event` during review.
R30	BBP shall create an `Obstacle_report` associated with the trip only for confirmed `sensor_event`s.

Continued on next page

ID	Description
R31	After recording (and review), BBP shall calculate final trip statistics. The 'Average Speed' (avg_speed) shall be calculated using 'Elapsed Time' and 'Total Distance'.
R32	After recording, BBP shall retrieve and associate weather information with the trip.
R33	BBP shall allow RCY to save the completed `Trip` (path, stats, weather, confirmed reports) to their private archive.
R34	BBP shall allow an RCY to view their list of saved trips.
R35	BBP shall allow an RCY to view the details of a saved trip.
R36	BBP shall provide an editor for RCY to manually create a path geometry on a map.
R37	BBP shall allow RCY to submit `Status_report`s manually for paths.
R38	BBP shall allow RCY to submit `Obstacle_report`s manually for paths.
R39	BBP shall allow an RCY, during the save process (UC9, UC12), to specify: (1) The visibility of the `Path` (Public/Private for search), and (2) The visibility of the `Trip` (Public/Private for statistics aggregation).
R40	BBP shall display feedback messages (confirmations, errors) to the user.

3.2.2. Use case diagrams

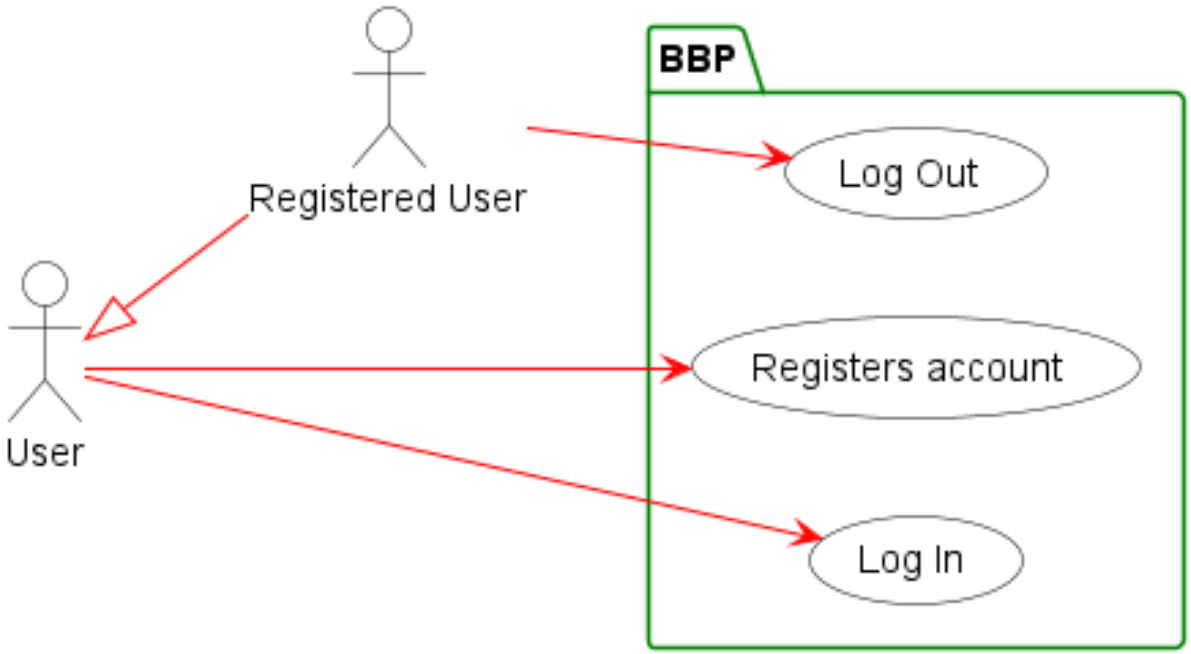

Figure 3.5: Use Cases Diagram for Users.

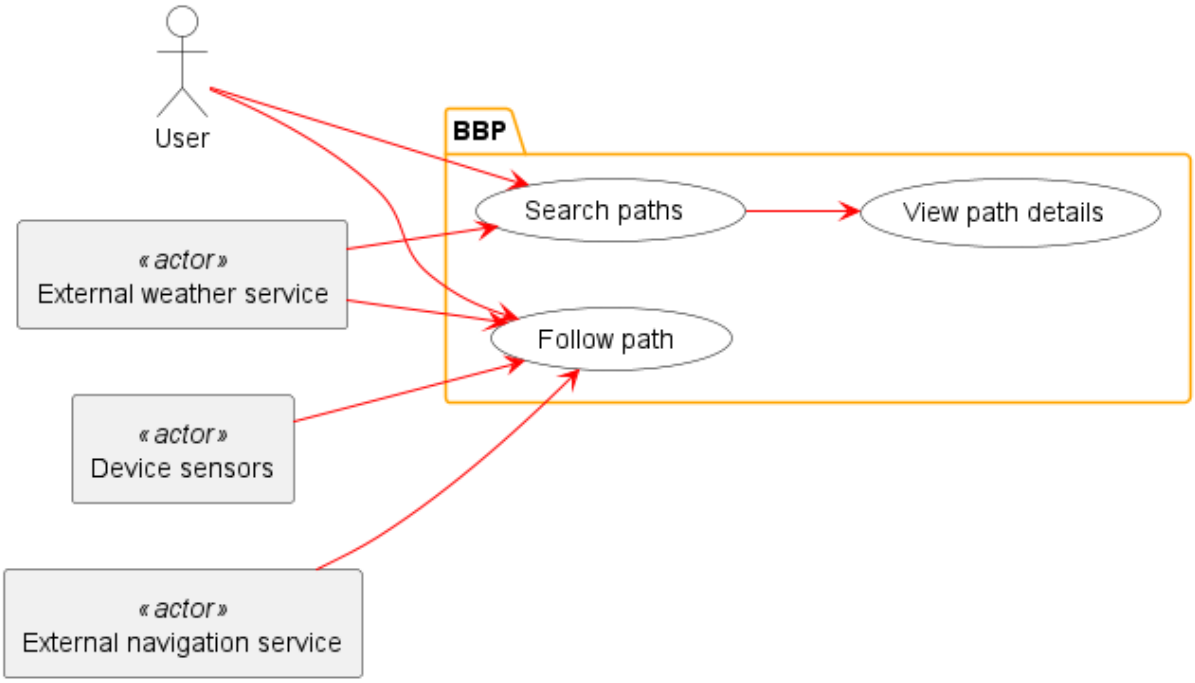

Figure 3.6: Use Cases Diagram for both ACY and RCY (general user).

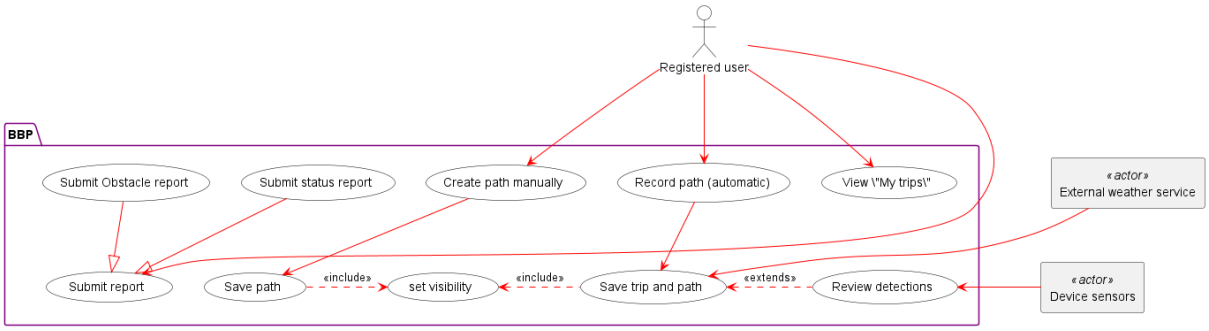

Figure 3.7: Use Cases Diagram for RCY

Use cases and sequence diagrams

In this section, they are explained and represented the main identi ed use cases. There is a table with entry conditions, event ow, exit conditions and exception for each of them, and a sequence diagram that shows the messages exchanged between the entities and the called functions

[UC1]. Register Account

Actor	**User**
Entry conditions	**The User is unauthenticated** and is on a screen with a "Register" option.
Event Flow	1- BBP shows the main screen with a "Register" button. (R1) 2- The User taps the "Register" button. 3- BBP shows the registration form. 4- The User inserts their username, email, and password in the form. 5- The User taps the "Submit" button. 6- BBP checks all credentials (e.g., uniqueness, password matching).(R2) 7- BBP creates the new Registered User account in the system. 8- BBP automatically logs in the user and redirects to the main map screen.
Exit condition	The user is authenticated as a Registered User and is on the main map screen.
Exceptions	• The email address is already linked to an account. An error message is shown, and the user remains on the registration form. • The username is already taken. An error message is shown, and the user remains on the registration form. • Invalid password (e.g., does not meet complexity requirements). An error message is shown, and the user remains on the registration form.

Table 3.2: Register Account use case.

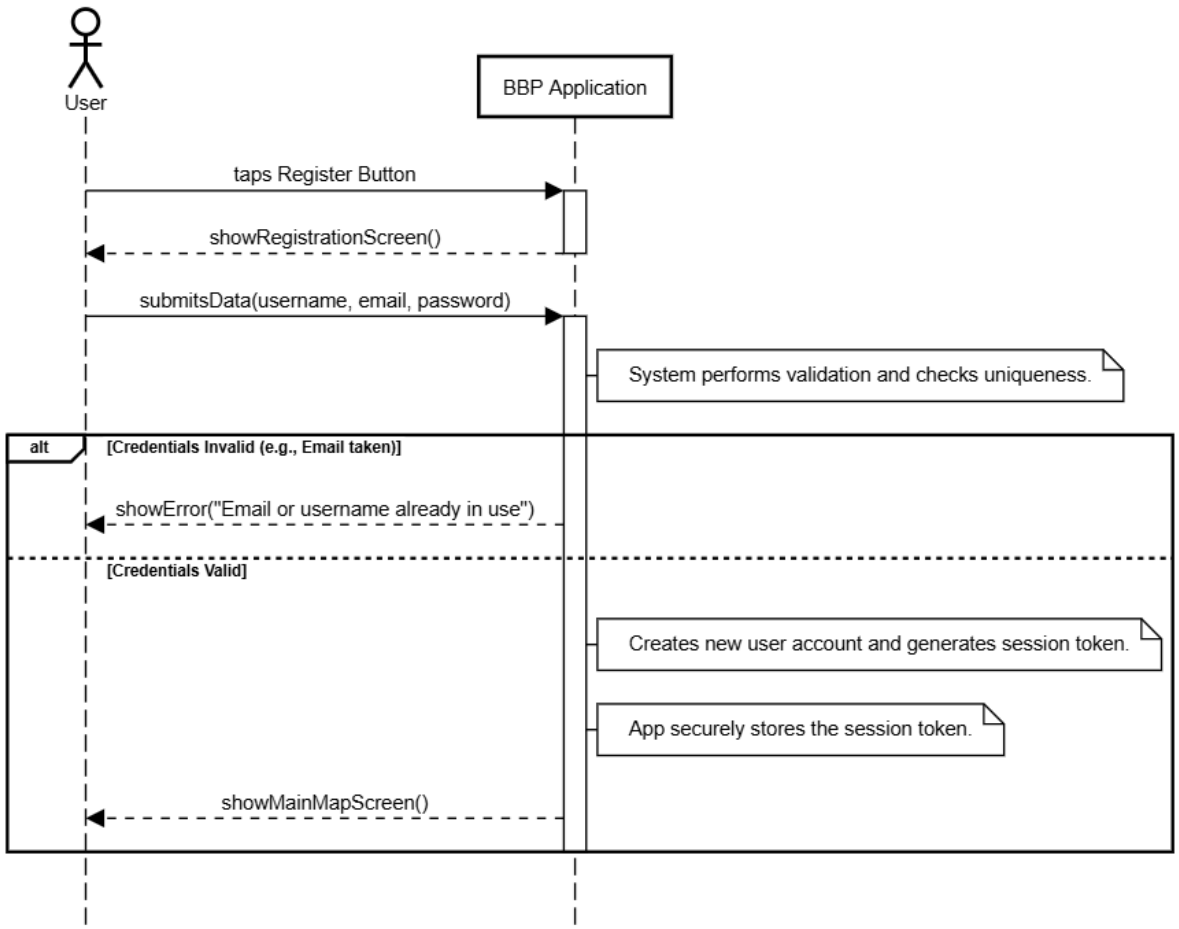

Figure 3.8: UC1

[UC2]. Log In

Actor	**User**
Entry conditions	**The User is unauthenticated** and is on a screen with a "Log In" option.
Event Flow	1 - BBP shows the login form.(R3) 2 - The User inserts their username (or email) and password. 3 - The User taps the "Log In" button. 4 - BBP checks the credentials against the database.(R4) 5 - BBP authenticates the user and creates an access session.
Exit condition	The user is authenticated as a Registered User and is redirected to the main map screen.

Exceptions	• The provided credentials (username/password) are incorrect. An error message is shown, and the user remains on the login page.

Table 3.3: Log In use case.

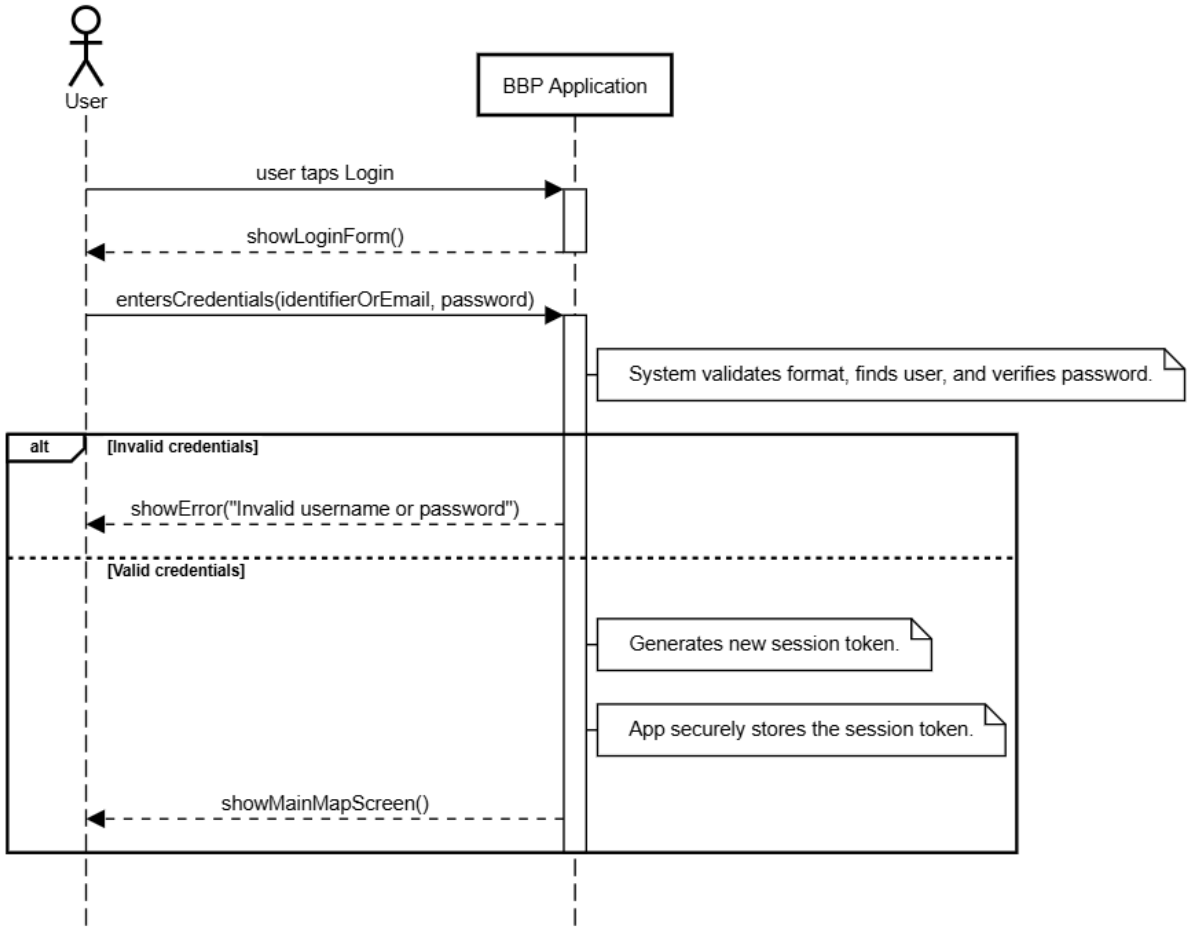

Figure 3.9: UC2

[UC3]. Log Out

Actor	Registered User
Entry conditions	The user is currently authenticated (logged in) in the BBP system.
Event Flow	1 - The User navigates to their profile or settings menu. 2 - The User taps the "Log Out" button.(R6)

	3 - BBP invalidates and terminates the user's session. 4 - BBP redirects the user to the main map screen (now as a **User**).
Exit condition	The user's session is terminated, and they are now a **User**.
Exceptions	• None.

Table 3.4: Log Out use case.

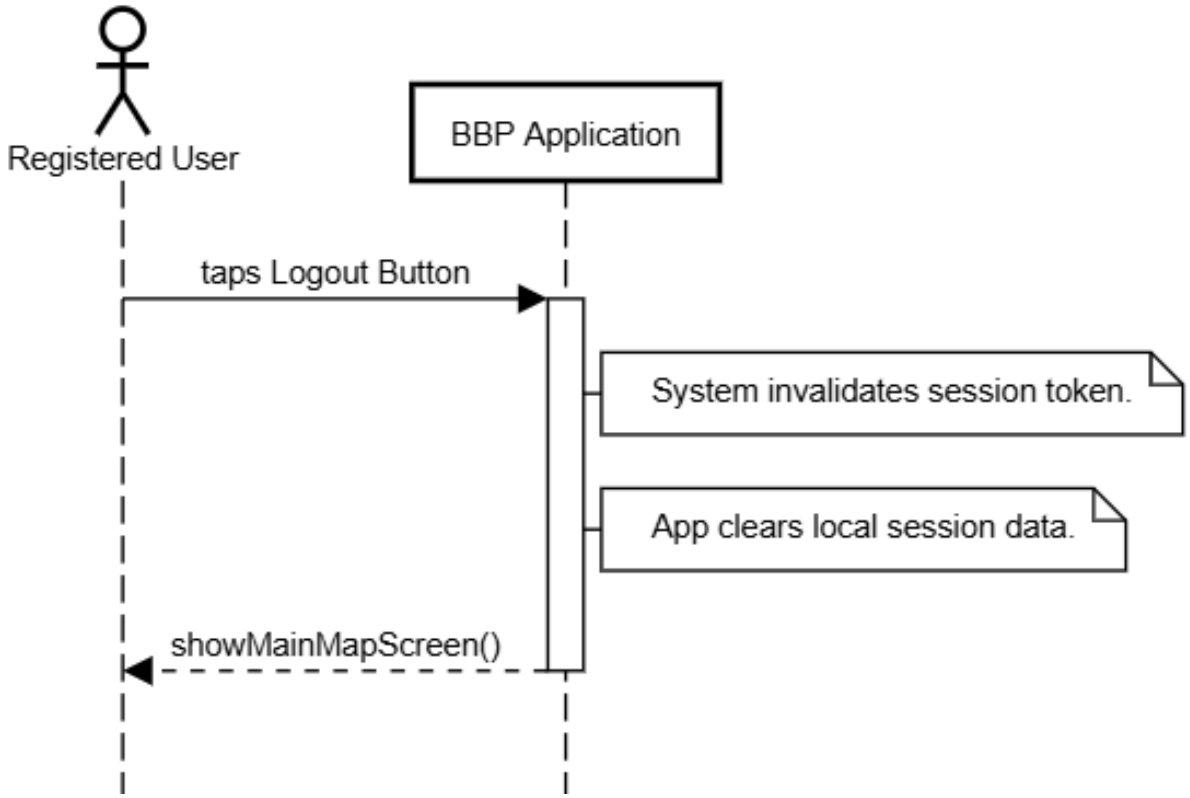

Figure 3.10: UC3.

[UC4]. Search Paths

Actor	**User**, External Weather Service
Entry conditions	The user is on the main screen of the application.
Event Flow	1 - The User enters an origin and a destination in the search bar. 2 - The User taps the "Search" button (R7). 3 - **BBP processes the query by:**

	a. Identifying all relevant public `Paths` (checking Visibility R25). **b. Retrieving all associated `Status_report` and `Obstacle_report` instances (R8).** **c. Applying the merging (R9) and scoring (R10) logic to calculate the 'derivedStatus' and 'derivedScore' for each path.** **d. Querying the `External Weather Service` for forecast data for the relevant locations (R14).** 4 - BBP displays the ordered list of path to the User (R11), including their status and weather info (R14).
Exit condition	The user is presented with a list of matching paths ordered by score, or a "No results" message.
Exceptions	• No paths are found matching the query. The system displays a "No results found" message. • The `External Weather Service` is unavailable. Paths are shown without weather data, and a "Weather unavailable" message is displayed.

Table 3.5: Search Paths use case.

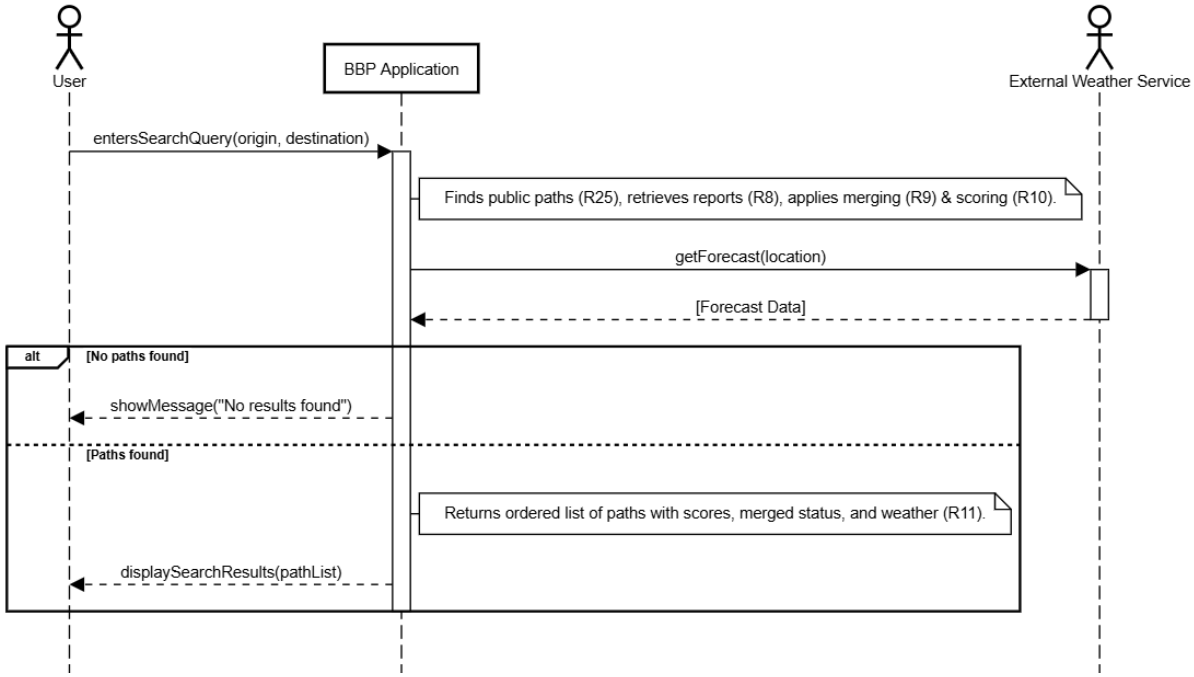

Figure 3.11: UC4.

[UC5]. View Path Details

Actor	**User**, External Weather Service
Entry conditions	The user is viewing a list of search results (`pathList`) from UC4.
Event Flow	1 - The User taps on a specific path from the results list (R12). 2 - **BBP retrieves the full details for the selected path by:** **a. Retrieving its complete list of `GPS_points`, all associated `Status_reports`, and all `Obstacle_reports` (R8).** **b. Applying the merging logic (R9) using the reports to calculate the definitive `derivedStatus`.** **c. Querying the `External Weather Service` for an updated forecast (R14).** 3 - **BBP Application displays the `Path Details` screen to the User**, showing all path details (geometry, inclination, merged status, obstacles, weather) (R13)and the weather forecast for the path location (R14).
Exit condition	The user is viewing the detailed information for the selected path.

Exceptions	• The selected `pathID` is invalid or no longer available. The system shows an error message. • The `External Weather Service` is unavailable. Path details are shown without weather data.

Table 3.6: View Path Details use case.

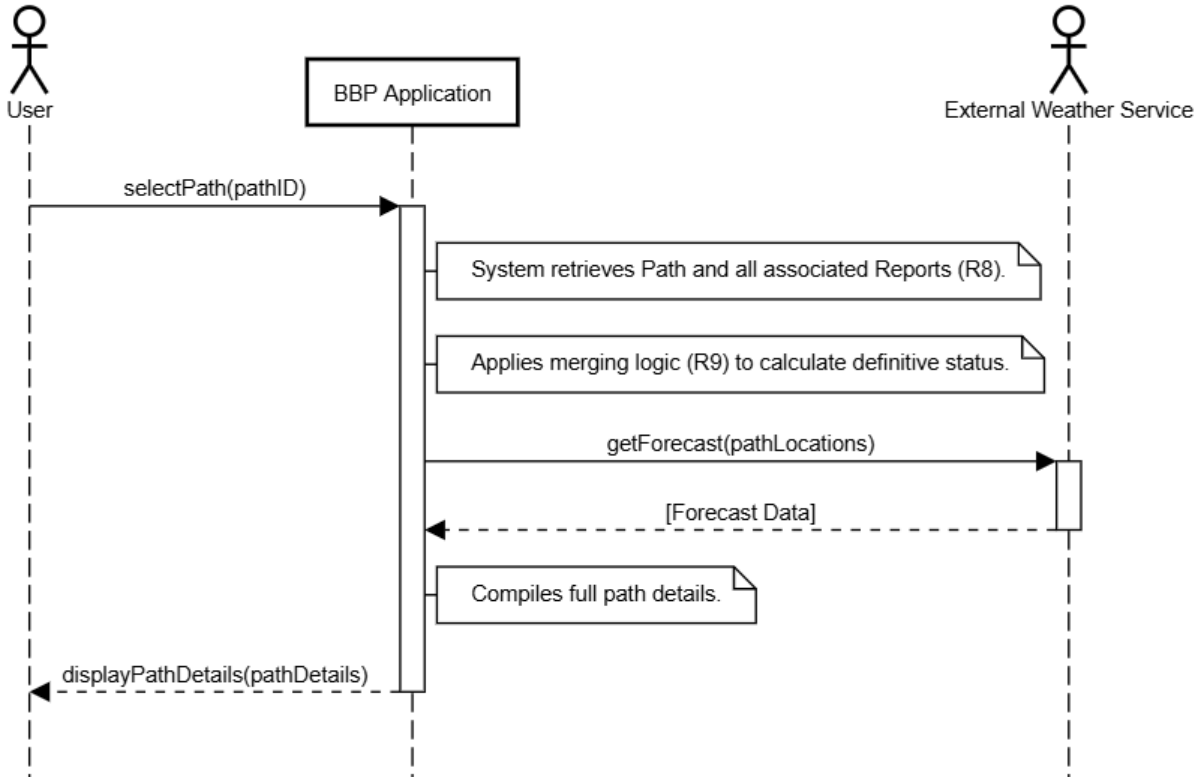

Figure 3.12: UC5 View Path Details

[UC6]. Follow Path (Navigation)

Actor	**User**, External navigation service, Device Sensors (GPS)
Entry conditions	The user is viewing the `Path Details` screen (from UC5).
Event Flow	1 - The User taps the "Follow Path" button (R15). 2 - BBP Application sends the path geometry to the device's External navigation service (SP22) and simultaneously activates its background statistics tracking (R15).

	3 - The External navigation service displays the turn-by-turn route to the User. 4 - Concurrently, the BBP Application requests continuous location updates from the Device Sensors (GPS) in the background. 5 - **Note:** The BBP app does not record the full geometry (it's already known) and does not display a map, as the user is viewing the external app. 6 - The User arrives at the destination (or manually closes the external navigator and returns to the BBP Application to tap "Stop Navigation") (SP23). 7 - BBP Application stops the location updates and finalizes the session statistics using 'Elapsed Time' (R31). 8 - BBP Application displays a "Summary" screen showing the session statistics (time, distance, avg speed) (R16). 9 - **If User is unauthenticated:** The Summary screen includes a "Register to Save" button (R17, R18). 10 - The **unauthenticated User** taps "Register to Save", completes registration (UC1), selects a visibility for the Trip (R39.2), and the activity (stats-only) is saved (R19). 11 - **If User is Registered:** The Summary screen includes a "Save Activity" button. 12 - The Registered User taps "Save Activity", selects a visibility for the Trip (R39.2), and the activity (stats-only) is saved to their archive (R33). 13 - If the user (**unauthenticated** or Registered) exits the summary screen without saving/registering, the session data is discarded (R18).
Exit condition	The user has completed navigation. The session activity (stats-only) is either saved with a visibility setting or discarded.
Exceptions	• The user cancels navigation mid-way (by returning to BBP and pressing "Stop"). The flow stops, stats are discarded, and the user is returned to the `Path Details` screen.

Table 3.7: Follow Path (Navigation) use case.

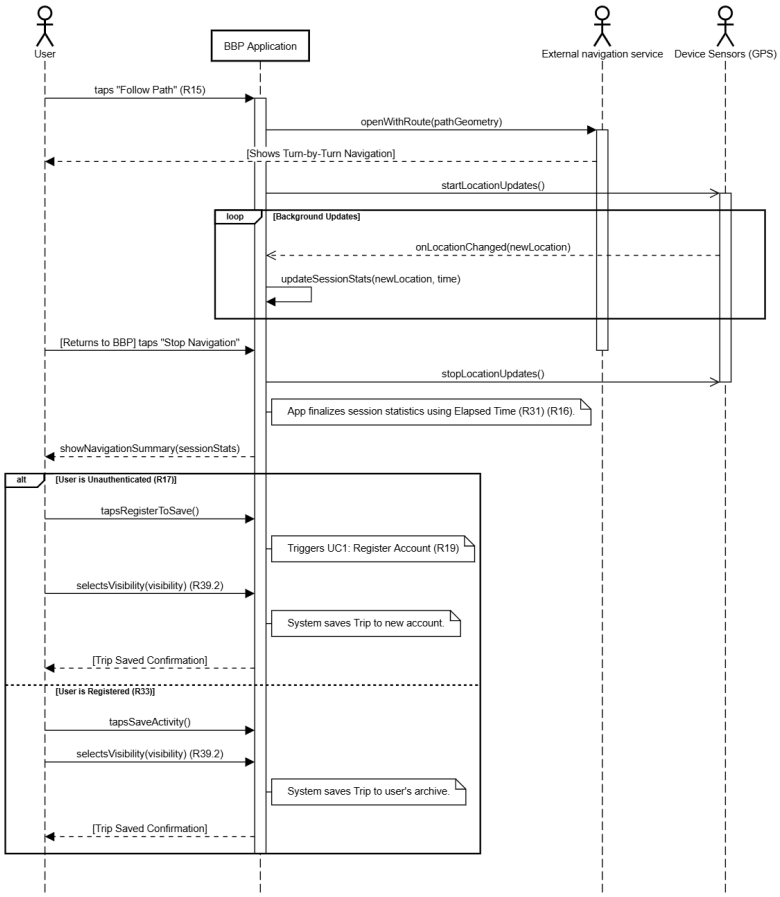

Figure 3.13: UC6 Follow Path (Navigation)

[UC7]. Record Path (Automatic)

Actor	Registered User, Device Sensors (GPS, IMU)
Entry conditions	The user is registered and logged in. (Optionally, the user may have selected a `Path` to follow).
Event Flow	1 - The User taps the "Start Recording" button (R20). 2 - BBP activates `Device Sensors` (R23). 3 - BBP displays the recording HUD, showing real-time statistics (R22). 4 - **In parallel:** BBP records the user's `GPS_points` sequence (R21) and monitors sensor data (R23). 5 - **If** sensor data exceeds the threshold, BBP internally logs a new `sensor_event` (R24). 6 - The User taps the "Stop Recording" button (R27).

	7 - BBP stops all sensors and finalizes the `Path` geometry. 8 - BBP calculates the final `Statistics` using 'Elapsed Time' and 'Total Distance' (R31). 9 - BBP triggers the **'Save Trip and Path'** use case (UC9), optionally displaying *ReviewDetections* (UC8) if events were logged.
Exit condition	The user has completed the recording. The system proceeds to the save flow (UC9).
Exceptions	• The user discards the trip instead of saving. The recorded data is discarded.

Table 3.8: Record Trip (Automatic) use case.

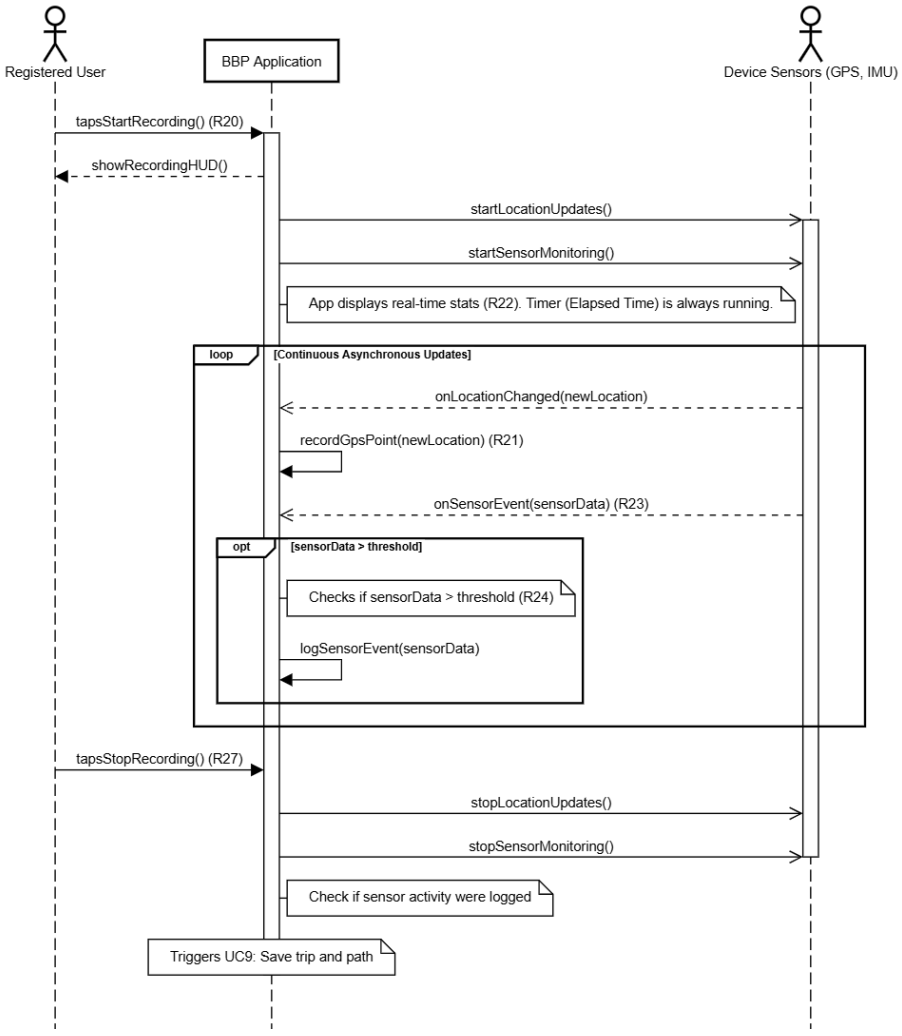

Figure 3.14: UC7 Record Trip

[UC8]. Review Detections

Actor	Registered User
Entry conditions	The user has just entered `Save Path and Trip` (UC9). BBP has logged one or more `sensor_events` during the trip (R24).
Event Flow	1 - BBP displays the trip summary with the `Review Detections` screen. 2 - BBP shows the recorded path on a map, with pins marking the location of each logged `sensor_event`. (R28) 3 - The User reviews each detection (e.g., by tapping the pin). 4 - For each event, the User chooses "Confirm" or "Ignore" (R29). 5 - **If** "Confirm" is pressed, BBP flags the `sensor_event` as confirmed, ready to become an `Obstacle_report` (R30). 6 - **If** "Ignore" is pressed, BBP flags the `sensor_event` as rejected.
Exit condition	All pending `sensor_events` have been processed. The system proceeds to the `Save Path and Trip` (UC9) decision.
Exceptions	• The user exits the review screen prematurely. BBP treats all pending detections as "Ignored".

Table 3.9: Review Detections use case.

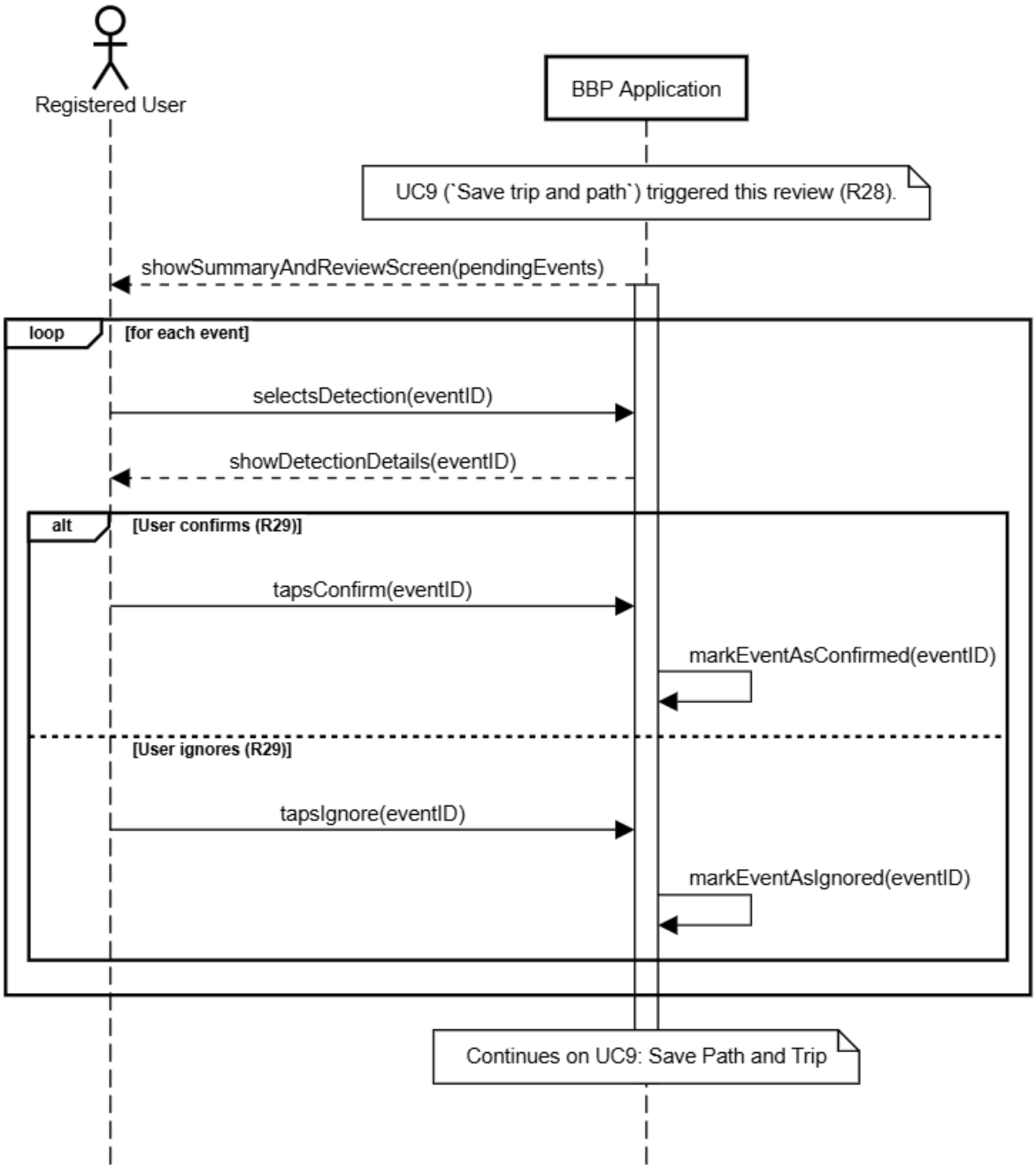

Figure 3.15: UC8 Review Detections

[UC9]. Save trip and path

Actor	Registered User, External Weather Service
Entry conditions	The user has just completed `Record Trip (Automatic)` (UC7).

	BBP has the complete recorded data (geometry, stats, and pending `sensor_events`).
Event Flow	1 - (Triggered by UC7) BBP retrieves weather data (R32) and displays the "Summary & Save" screen, showing the map and final statistics (R31). 2 - **Extension Point 'Review Detections' (UC8): If** pending `sensor_events` exist (R28), the flow is extended by UC8 to confirm/ignore detections. 3 - The User enters a name for the activity. 4 - **'«include»' 'set visibility' (UC10):** The User selects the visibility for the **'Trip' (Public/Private for stats [R26])** and for the **'Path' (Public/Private for search [R25])**. 5 - The User taps "Save" or "Discard". 6 - **If** "Save" is pressed: a. BBP saves the `Trip` to the user's private archive (R33). b. BBP saves the `Path` with its chosen visibility (R25). c. **If** the `Trip` visibility was "Public", BBP updates the `Path`'s average statistics (R9, R26). d. BBP displays a success message (R40). 7 - **If** "Discard" is pressed, the flow terminates and data is deleted.
Exit condition	The new `Trip` is saved. The new/updated `Path` is saved. Or all data is discarded.
Exceptions	• Invalid data (e.g., missing name). The system shows an error and remains on the save screen.

Table 3.10: Save trip and path use case.

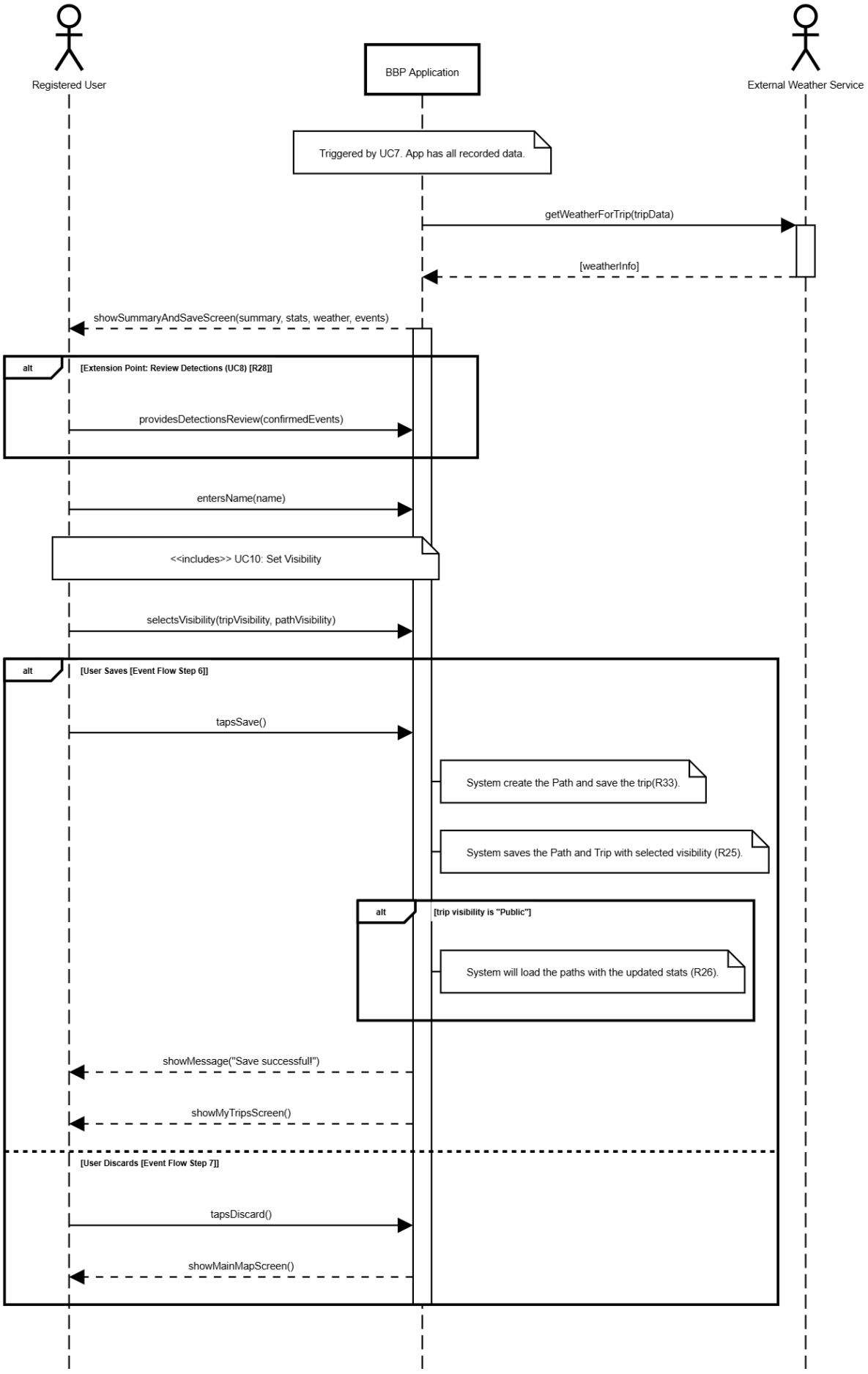

Figure 3.16: UC9 Save Path and Trip

[UC10]. set visibility (Included)

Actor	Registered User
Entry conditions	The user is in the process of saving an activity (called by UC9 or UC12).
Event Flow	1 - (Triggered by UC9 or UC12) BBP presents the User with the relevant visibility options [R25, R26]. 2 - **If** called by UC9 (`Save trip and path`), the User selects visibility for both the `Trip` (stats aggregation [R26]) and the `Path` (search [R25]). 3 - **If** called by UC12 (`Save path`), the User selects visibility only for the `Path` (search [R25]). 4 - The User confirms the selection (R40).
Exit condition	The visibility preference(s) are set and returned to the calling use case.
Exceptions	• None. A default selection (e.g., "Private") is assumed.

Table 3.11: set visibility (Included) use case.

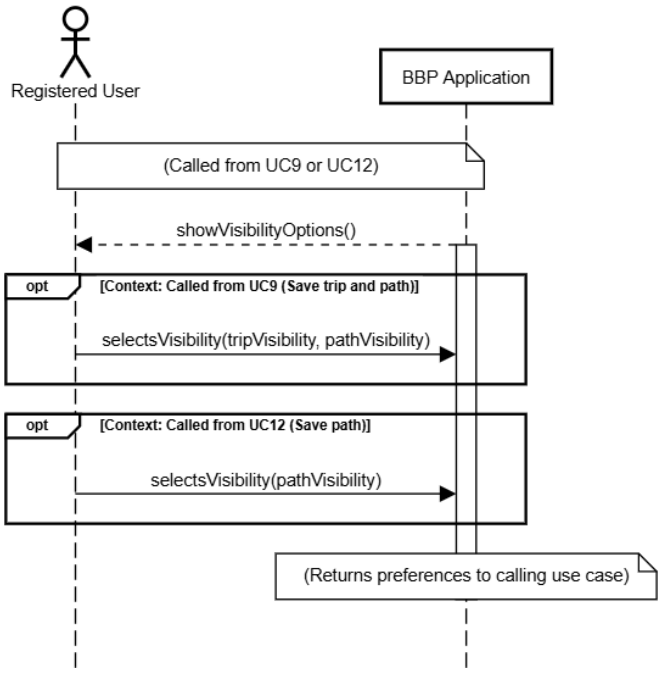

Figure 3.17: UC10 set visibility

[UC11]. Create path manually

Actor	Registered User
Entry conditions	The user is registered, logged in, and has selected the "Create Path Manually" option.
Event Flow	1 - BBP displays the manual path editor screen, showing a map. 2 - The User iteratively draws the path geometry by placing points on the map (R36). 3 - The User taps "Done" or "Finish" when the path geometry is complete. 4 - The Registered User can now add a path status or report an Obstacle (UC14, UC15). 5 - BBP proceeds to the **'Save path'** use case (UC12), passing the new path data.
Exit condition	The user has finished creating the path geometry. The system proceeds to the save flow (UC12).
Exceptions	• The user cancels creation mid-way. The unsaved path data is discarded.

Table 3.12: Create path manually use case.

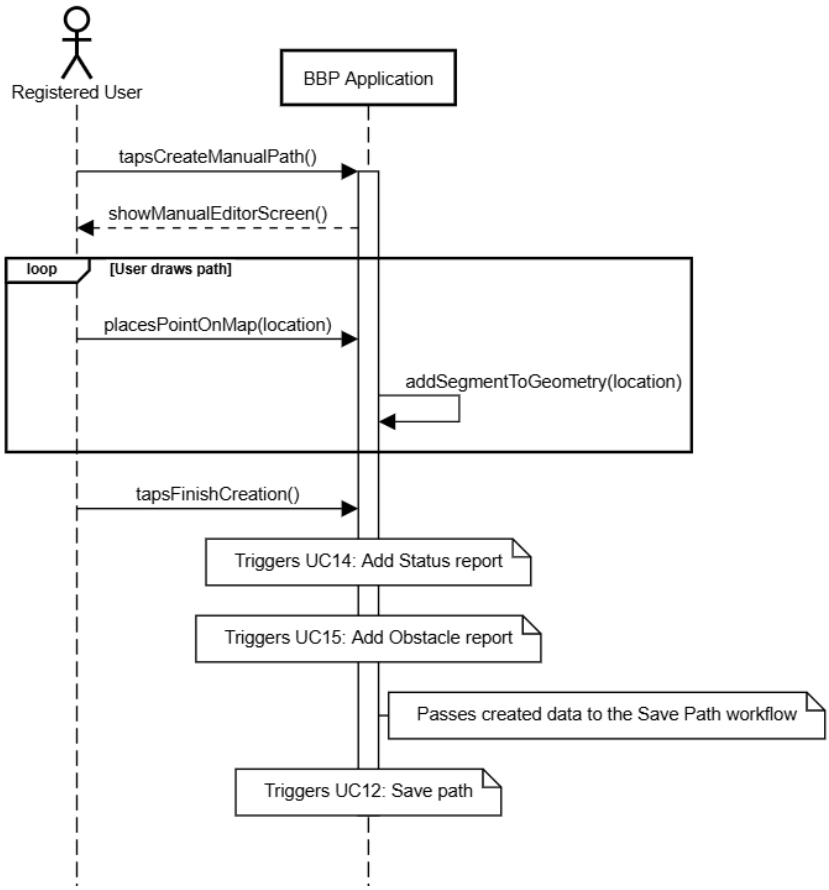

Figure 3.18: UC11 create path manually

[UC12]. Save path

Actor	Registered User
Entry conditions	The user has just completed `Create path manually` (UC11). BBP has the new path geometry and any manual reports.
Event Flow	1 - (Triggered by UC11) BBP displays the "Save Path" screen. 2 - The User enters a name for the `Path`. 3 - **‘«include»‘ ‘set visibility‘ (UC10):** The User selects the visibility for the **‘Path‘** (e.g., "Public" or "Private") [R25]. 4 - The User taps "Save" or "Discard". 5 - **If** "Save" is pressed, BBP saves the new `Path` with its chosen visibility (R25). 6 - **If** "Discard" is pressed, the flow terminates and data is deleted.

Exit condition	The new `Path` is saved with the chosen visibility, or all data is discarded.
Exceptions	• Invalid data (e.g., missing name). The system shows an error and remains on the save screen.

Table 3.13: Save path use case.

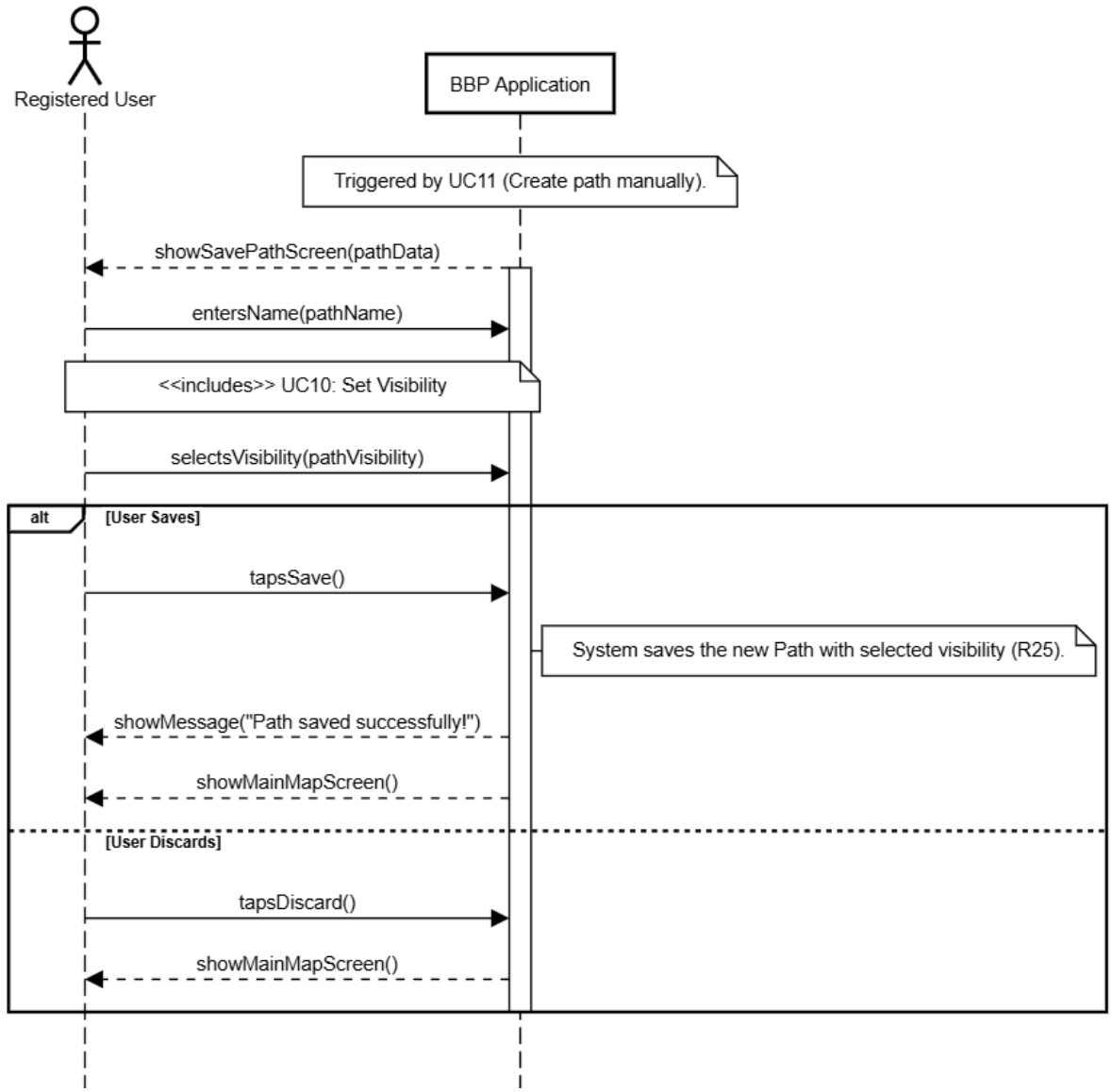

Figure 3.19: UC12 save path

[UC13]. View "My trips"

Actor	Registered User
Entry conditions	The user is registered and logged in.
Event Flow	1 - The User navigates to their profile and taps the "My Trips" option. 2 - **BBP retrieves all `Trip` objects associated with that user (R34) and displays a summary list (e.g., name, date, distance).** 3 - The User taps on a specific `Trip` in the list. 4 - **BBP retrieves the full `Trip` object, including its `GPS_points`, `Statistic`, `Weather_info`, and any associated `Obstacle_reports` (R35).** 5 - BBP displays the detailed trip summary to the User.
Exit condition	The user has viewed their list of trips or the details of a specific trip.
Exceptions	• The user has no saved trips. The system displays an empty list or a "No trips recorded" message.

Table 3.14: View "My trips" use case.

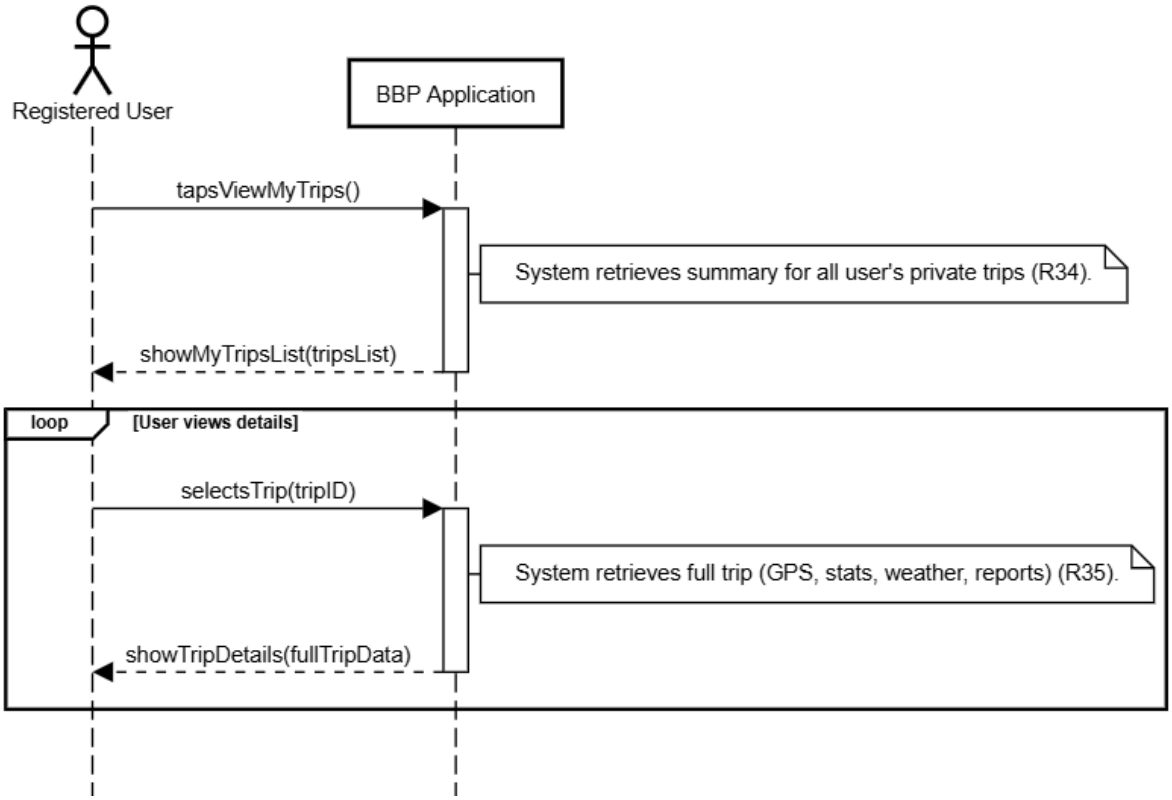

Figure 3.20: UC13 view "My trips"

[UC14]. Submit status report

Actor	Registered User
Entry conditions	The user is registered, logged in, and is viewing the details of a `Path` (UC5) or is adding a `Path manually` (UC11).
Event Flow	1 - The User taps the "Report status" button. 2 - BBP displays the status options (e.g., "Optimal", "Requires Maintenance"). 3 - The User selects a status and taps "Submit". 4 - **BBP saves the new `Status_report` associated with the current `Path` (R37).** 5 - **BBP triggers a recalculation of the `Path`'s 'derivedStatus' (R9) and 'derivedScore' (R10).** 6 - **BBP displays a success message to the User.**

Exit condition	The new `Status_report` is saved. The `Path`'s status and score are updated. The user is returned to the `Path Details` screen or to the `Save path screen`.
Exceptions	• The user cancels the submission. No report is created.

Table 3.15: Submit status report use case.

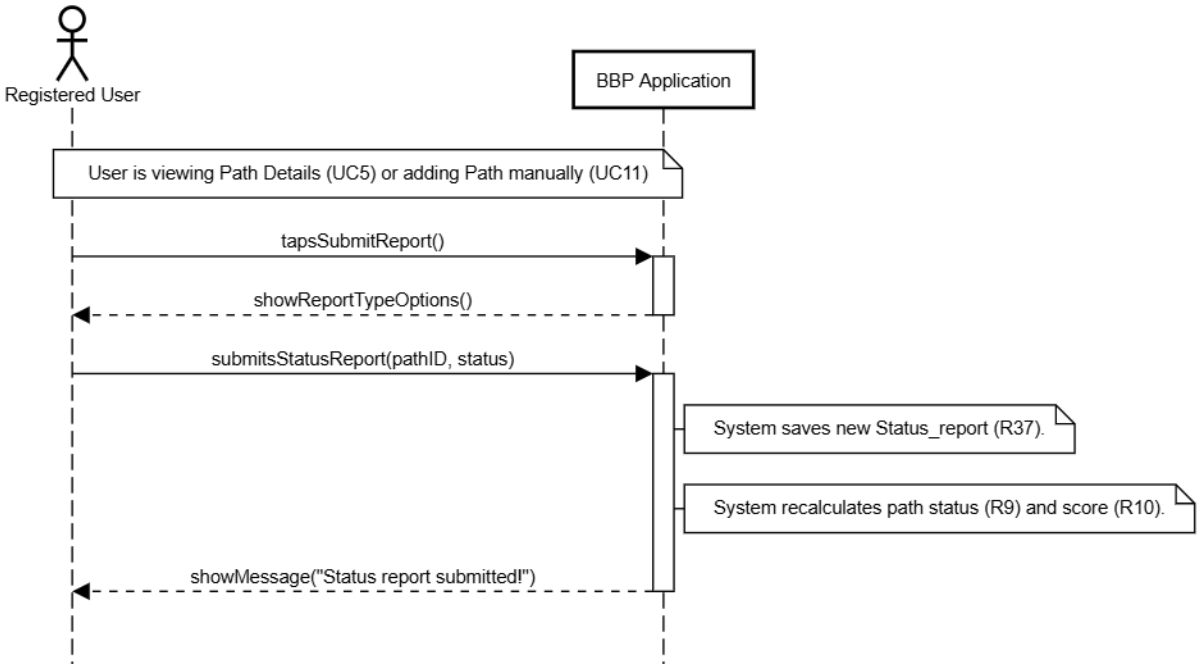

Figure 3.21: UC14 submit status report

[UC15]. Submit Obstacle report

Actor	Registered User
Entry conditions	The user is registered, logged in, and is viewing the details of a `Path` (UC5) or is adding a `Path manually` (UC11).
Event Flow	1 - The User selects "Submit Obstacle" (R38). 2 - BBP displays the obstacle report form (e.g., type of obstacle, location pin). 3 - The User selects an obstacle `type` (e.g., "Pothole"), places a pin on the map to indicate the location, and taps "Submit".

	4 - **BBP saves the new `Obstacle_report` associated with the current `Path` (R38).** 5 - **BBP triggers a recalculation of the `Path`'s 'derivedScore', as obstacles affect the score (R10).** 6 - **BBP displays a success message to the User.**
Exit condition	The new `Obstacle_report` is saved. The `Path`'s score may be updated. The user is returned to the `Path Details` screen or to the `Save path screen`.
Exceptions	• The user cancels the submission. No report is created. • Invalid data (e.g., missing location). The system shows an error.

Table 3.16: Submit Obstacle report use case.

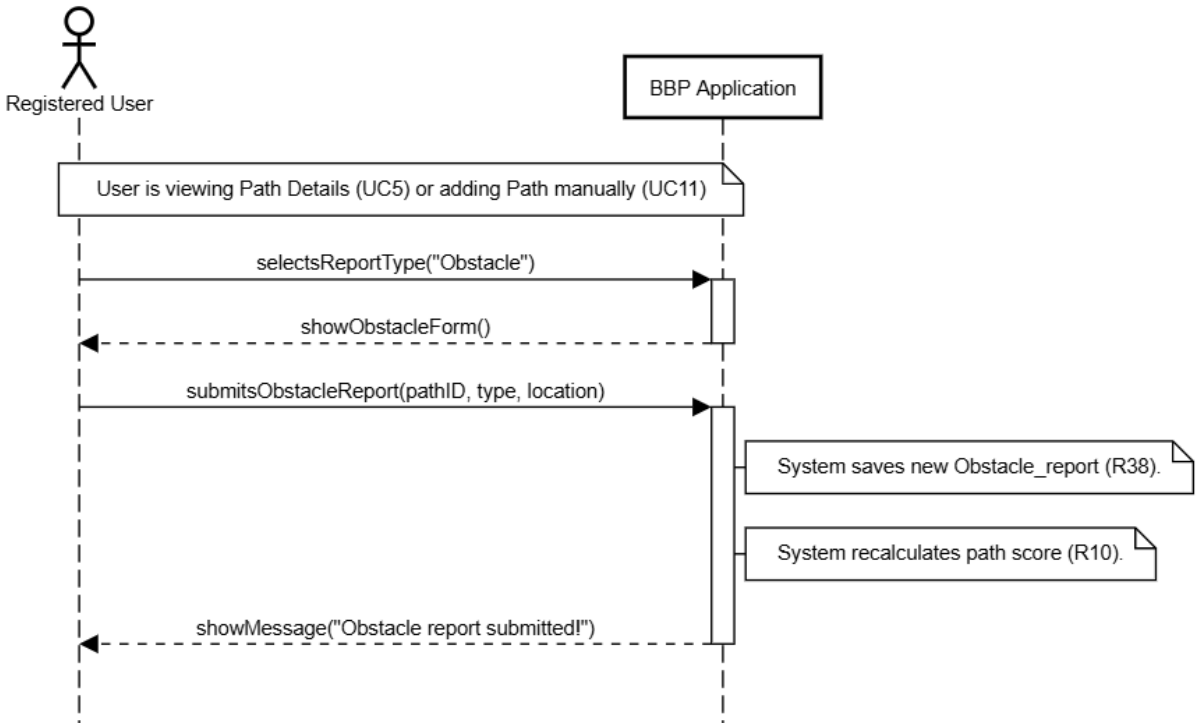

Figure 3.22: UC15 submit Obstacle report

3.2.3. Mapping on goals

[G1] Support personal cycling activity tracking

Requirements	Domain Assumptions
R1-R6: Authentication and Profile Management requirements (Signup, Login, View Personal Page, Logout).	DA1: Users must have a mobile device with GPS, accelerometer, and gyroscope sensors.
R15: BBP shall allow any user to initiate a navigation mode ("Follow Path") for a selected path.	DA2: The GPS provides location data accurate enough to reconstruct the path.
R16: Upon completion of navigation, BBP shall show trip summary (avg. speed, distance, ...).	DA3: Accelerometer and gyroscope data correlate with real-world events.
R19: BBP shall allow anonymous users who accept registration to save the navigated session as a Trip.	DA4: A threshold exists to distinguish potholes from vibrations.
R20: BBP shall allow an RCY to start an automatic path recording session.	DA5: The device remains physically attached to the user or bike.
R21: During automatic recording, BBP shall record the user's GPS coordinates.	DA7: Users act in good faith when providing or confirming data.
R22: During automatic recording, BBP shall display real-time statistics.	DA11: The external weather API is stable and accurate.
R27: BBP shall allow an RCY to stop the automatic trip recording.	DA12: The user will not change the track or get lost.
R31: BBP shall calculate final trip statistics (Elapsed Time, Average Speed).	
R32: BBP shall associate weather data with the trip.	
R33: BBP shall allow the user to save the completed Trip to their archive.	
R40: BBP shall display feedback messages (confirmations, errors).	

[G2] Gather and provide actionable data on cycle path conditions

Requirements	Domain Assumptions
R7: BBP shall allow users to search for public bike paths by origin/destination.	DA1: Users must have a mobile device with GPS, accelerometer, and gyroscope.
R8: BBP shall retrieve all relevant reports for found paths.	DA2: GPS accuracy sufficient to reconstruct the path.
R9: BBP shall calculate merged status using freshness and majority rules.	DA3: Sensor data correlate with real-world events.
R10: BBP shall calculate a score for each path.	DA4: Threshold distinguishes potholes from normal vibrations.
R11: BBP shall display search results as a list of paths ordered by the calculated score.	DA5: Device physically attached to user or bike.
R12: BBP shall allow any user to select a path from the search results to view its details.	DA6: Device has enough resources (battery, CPU).
R13: BBP shall display the details of a selected path, including its geometry (map), distance, elevation, calculated merged status, and reported obstacles.	DA7: Users act in good faith when providing data.
R14: BBP shall retrieve and display current weather forecast information for the path location(s).	DA8: Recent data are considered more relevant.
R16: BBP shall allow any user to initiate a navigation mode ("Follow Path") for a selected path.	DA9: Users provide rational path evaluations.
R17: Upon completion of navigation by a user, BBP shall show the summary of the trip.	DA11: External weather API stable and accurate.
R20: BBP shall allow anonymous users who accept registration to save the navigated session as a Trip.	DA12: User will not change the path.
R21: BBP shall allow an RCY to start an automatic path recording session.	DA13: Active internet connection when searching/saving.

R22: BBP shall record GPS coordinates.	DA14: Paths correspond to safe cycling paths.
R23: BBP shall display real-time statistics.	
R35: BBP shall allow an RCY to view their list of saved trips.	
R36: BBP shall allow an RCY to view the details of a saved trip.	

[G3] Facilitate the discovery and navigation of suitable cycling paths

Requirements	**Domain Assumptions**
R14: BBP shall retrieve and display weather for selected path.	DA2: GPS accuracy sufficient to reconstruct the path.
R15: BBP shall allow navigation for selected path.	DA12: The user will not change the track or get lost.
R16: BBP shall show trip summary.	
R17: BBP shall offer registration to anonymous users after navigation.	
R18: If registration refused, data are discarded.	
R19: If accepted, session is saved as Trip.	

[G4] Enhance trip analysis with meteorological and physical data

Requirements	**Domain Assumptions**
R31: BBP shall calculate final trip statistics (Average Speed, Elapsed Time).	DA11: The external weather API is stable and accurate.
R32: BBP shall retrieve and associate weather data with the trip.	
R33: BBP shall allow the user to save the completed Trip.	

[G5] Establish a community-based path evaluation system

Requirements	Domain Assumptions
R25: BBP shall store the visibility setting of a saved Path as specified by the Registered User, ensuring that Public paths are retrievable via the search function (R7), while Private paths remain strictly accessible only within the creator's personal archive.	DA1: Users must have a mobile device with sensors (GPS, accelerometer, gyroscope).
R26: BBP shall store the visibility setting of a saved Trip as specified by the Registered User, ensuring that data from Public trips are included in the aggregated Path statistics and scoring algorithms (R9, R10), while data from Private trips are excluded from all community calculations.	DA2: GPS accuracy sufficient to reconstruct the path.
R36: BBP shall provide a map editor for RCY to create paths.	DA9: Users provide correct and rational evaluations.
R37: BBP shall allow RCY to submit Status reports.	DA10: Users can correctly interpret path status levels ("Optimal", "Requires Maintenance").
R38: BBP shall allow RCY to submit Obstacle reports. R39: BBP shall allow RCY to set visibility options for Path and Trip. R40: BBP shall display confirmations or errors.	

Requirements	G1	G2	G3	G4	G5
R1	✓				
R2	✓				

R3	✓				
R4	✓				
R5	✓				
R6	✓				
R7	✓	✓			
R8		✓			
R9		✓			
R10		✓			
R11		✓			
R12		✓			
R13		✓			
R14		✓	✓		
R15	✓	✓	✓		
R16	✓	✓	✓		
R17			✓		
R18			✓		
R19	✓	✓	✓		
R20	✓	✓			
R21	✓	✓			
R22	✓	✓			
R23		✓			
R24		✓			
R25					✓
R26					✓
R27	✓	✓			
R28		✓			
R29		✓		✓	
R30		✓		✓	
R31	✓	✓		✓	
R32	✓				

R33	✓				
R34		✓			
R35		✓			
R36					✓
R37					✓
R38					✓
R39					✓
R40	✓				✓

Table 3.22: Traceability Matrix for Goals and Requirements

3.3. Performance requirements

This section specifies the non-functional requirements related to system response time, concurrent user load, data storage capacity, and data acquisition timing constraints.

PR-1. (Time response: Login): The system shall authenticate a `RCY` (Registered Cyclist) (R3) and display the main map screen within **2.0 seconds** of the login submission.

PR-2. (Time response: Search): The system shall process a `Search paths` request (R7), including retrieving reports (R8), executing merging logic (R9), calculating scores (R10), fetching weather data (R14), and displaying the ordered results list (R11) within **5.0 seconds**.

PR-3. (Time response: View Details): The system shall display the full `View path details` screen (R13), including geometry, merged status, obstacles, and weather (R14), within **3.0 seconds** of the user's selection (R12).

PR-4. (Time response: Start Recording): The `Record trip (automatic)` session (R20) must initialize and begin active monitoring of sensors (R21, R23) within **1.5 seconds** of the user tapping "Start".

PR-5. (Time response: Sensor Frequency): During an **active** recording state (R20):

- The GPS (R21) must be sampled at a minimum frequency of **1 Hz** (one sample

per second) to ensure accurate geometry.
- The Accelerometer and Gyroscope (R23) must be sampled at a minimum frequency of **10 Hz** (ten samples per second) to reliably capture data for pothole detection (R24).

PR-6. (Time response: Save): The system shall complete the server-side save operation for a `Trip` (R33) and/or `Path` (R25), including persisting the selected visibility options (R39) and the stored visibility settings (R25, R26), and return a confirmation message (R40) within **4.0 seconds** of the user tapping "Save".

PR-7. (Number of concurrent users: General): The BBP backend shall support a minimum of **1,000 concurrent users** (ACY and RCY) performing searches (R7) or navigation (R15) without degrading the response times defined in PR-2 and PR-3.

PR-8. (Number of concurrent users: Recording): The system shall support at least **50 concurrent** `Record trip (automatic)` sessions (R20) being initiated per minute.

PR-9. (Data storage: Scalability): The system architecture must be scalable to support the storage and processing (R8, R9) of an average of **10,000 new** `Trips` and **50,000 new** `Reports` per day.

PR-10. (Data storage: Retention): All user-generated `Trip` data (R33),including final statistics (R31) and associated weather information (R32), `Path` data (R25), and `Report` data (R37, R38) shall be retained indefinitely.

PR-11. (Time response: Stop Recording): After the user taps "Stop Recording" (R27), BBP shall stop sensor acquisition and display the post-recording screen (Review Detections if `sensor_events` were logged (R28), otherwise the Trip summary) within **3 seconds**.

3.4. Design constraints

This section outlines specific constraints that restrict the design and implementation choices for the BBP system.

3.4.1. Standard compliance

- **Web Communication:** All communication between the BBP mobile application and the BBP backend server must use secure, standard protocols, specifically **HTTPS**.
- **Data Interchange:** Data interchange with all external services, particularly the `External Weather Service`, must adhere to standard data formats, primarily **JSON**.
- **Data Privacy:** As the system collects and stores sensitive user data, including personal information (R1, R2) and precise location history (R21, R33), the design must comply with relevant data protection regulations, such as the **GDPR**.

3.4.2. Hardware limitations

- **Device Type:** The system is constrained to run on consumer smartphones (e.g., iOS and Android) .
- **Required Sensors:** All core functionalities for automatic recording (R20, R23) are strictly dependent on the availability and user-granted permission for specific hardware sensors: **GPS**, **Accelerometer**, and **Gyroscope**. Devices lacking these sensors will be limited to manual path creation (R36) and search/navigation (R7, R15).
- **Connectivity:** The system is not designed for full offline operation. An active internet connection (Wi-Fi or cellular data) is a constraint for most core functions, including login (R3), search (R7), submitting Reports (R31, R38)and saving/publishing Trips and Paths (R33, R25, R39) with stored visibility rules (R25, R26).

Any other constraint

- **Platform Type:** The user interface must be implemented as a **native mobile application** to ensure reliable access to background processes and hardware sensors (GPS, IMU) . A web-based application is not sufficient to meet the core functional requirements (R20, R23).
- **External API Dependency:** The system is constrained by its dependency on third-

party services. The `External Weather Service` (R14, R32) and any underlying map provider (e.g., Google Maps, OpenStreetMap) are external dependencies. The design must be resilient to their potential downtime or API changes.

- **Architecture:** The requirement for server-side aggregation (R8, R9) and scoring (R10) constrains the system to a **Client-Server architecture**. A purely peer-to-peer or standalone application is not feasible.

3.5. Software system attributes

This section defines the key quality attributes (non-functional requirements) that the BBP system must exhibit.

3.5.1. Reliability

The system must demonstrate high reliability during its core operations.

- **Recording Stability:** The `Record trip (automatic)` session (R20) must be robust. The application must be able to handle continuous background data streams from GPS (R21) and IMU sensors (R23) for extended periods (e.g., up to 3 hours) without crashing or corrupting data.
- **Data Integrity (Sensors):** The system must reliably process sensor data (R23) to ensure that `sensor_event` logs (R24) are correctly generated based on the defined thresholds (DA4) and are not lost, even if the application is temporarily suspended.
- **Data Integrity (Save):** The system must ensure that a "Save" operation (Trip save: R33; Path save: R25; visibility selection: R39; stored visibility settings: R25, R26) is atomic. A failure during saving (e.g., network loss) must not result in partially saved or corrupted `Trip` or `Path` data.

3.5.2. Availability

The server-side components of the BBP system must be highly available.

- **Core Service Uptime:** All backend services responsible for user authentication (R3), path searching (R7), and data aggregation (R8, R9, R10) shall maintain an availability of **99.5%** (Tier 3 equivalent).
- **Graceful Degradation:** In the event of an `External Weather Service` (R14, R32) outage, the system shall remain available, displaying path and trip details without

weather data and notifying the user of the temporary unavailability.

3.5.3. Security

The system must protect the integrity and confidentiality of user data.

- **Data in Transit:** All communication between the BBP Application and the BBP Server must be encrypted using **HTTPS**.
- **Authentication:** User passwords must be stored on the server using a strong, salted, one-way hashing algorithm.
- **Authorization:** The system must enforce strict data access controls. A `RegisteredUser` must only be able to view their own private `Trip` data (R33), enforced by the trip visibility rules (R26), via the saved trips list/details features (R34, R35). Data associated with other users must not be accessible.
- **Data Privacy (Location):** User GPS data (R21), both in transit and at rest, must be treated as sensitive personal information and protected against unauthorized access.

3.5.4. Maintainability

The system must be designed to be easily modified and maintained.

- **Modularity (Scoring):** The logic for `Path` scoring (R10) and `Report` merging (R9) must be implemented in a distinct, loosely-coupled module on the server. This allows the aggregation algorithm to be tuned or replaced without impacting core trip recording or user management functionalities.
- **Modularity (Detection):** The algorithm for processing raw sensor data (R23) into `sensor_events` (R24) must be isolated within the client application, allowing the detection thresholds (DA4) to be updated or calibrated without requiring a full application rewrite.

3.5.5. Portability

- **Client Platform:** The BBP mobile application shall be designed to be portable across the two primary mobile operating systems: **iOS** and **Android**.
- **OS Dependencies:** The design must account for differences in OS-level APIs (e.g., for accessing sensor data (R23) and managing background processes (R20)), ensuring

functionally equivalent behavior on both platforms.

4 | Formal Analysis Using Alloy

4.1. Objectives of the analysis

The goal of this Alloy-based analysis is to complement the informal requirements and UML models with a precise, machine-checkable specification. We use Alloy to model a core subset of the BBP domain where inconsistencies are both likely and costly: (i) the structural relationships among users, paths, trips, sensor-detected events, and reports, and (ii) the rules that constrain visibility/privacy and the creation of validated obstacle reports.

In particular, the analysis focuses on two categories of properties.

First, we verify static consistency properties of the domain model: bidirectional associations (e.g., a trip belongs to exactly one path and a path collects its recorded performances), cardinalities, and key integrity constraints (e.g., the relationship between an `Obstacle_report` and the `sensor_events` that may have triggered it). This is important because the BBP data model combines multiple entities (Path, Trip, Report, `sensor_events`) with non-trivial constraints that can easily become contradictory when the system evolves.

Second, we validate privacy and lifecycle properties. We model visibility choices (Public/Private) and check that they enforce the intended privacy invariants, and we formalize the lifecycle of sensor events (Logged → AwaitingConfirmation → Confirmed/Ignored). This allows us to verify that only confirmed detections can become official obstacle reports, and that review states eventually lead to a decision.

To keep the model analyzable, we intentionally abstract away continuous or implementation-dependent aspects (e.g., geometric computations, exact GPS coordinates, numerical statistics, and external API details). These elements are represented as uninterpreted atoms, while the analysis concentrates on the logical constraints and state transitions that are central to correctness.

4.2. Alloy Code

```
// ----- 1. Enumerations -----

enum PathStatus { Optimal, Medium, RequiresMaintenance }
enum ObstacleType { Pothole, Roadwork, Other }
enum Visibility { Public, Private }
// sensor_event lifecycle from the RASD state diagram
enum DetectionState { Logged, AwaitingConfirmation, Confirmed, Ignored }

// ----- 2. Basic Data Types -----

sig GPS_point {}
sig Statistic {}
sig Weather_info {}
sig Timestamp {}

// ----- 3. Static Domain Model -----

abstract sig User {}

sig Registered_user extends User {
    trips: set Trip,            // Trips this user owns
    createdPaths: set Path,     // Paths this user created
    submittedReports: set Report // Reports this user authored
}

sig Path {
    geometry: set GPS_point,      // The GPS points defining the path
    pathVisibility: one Visibility,// Visibility for search (R25, R7) (choice in save
        flow: R39)
    creator: one Registered_user, // The user who created it
    reports: set Report,          // All reports for this path
    performances: set Trip        // All trips recorded on this path
}

// A specific, recorded cycling session
sig Trip {
    trace: set GPS_point,           // GPS trace of the ride (R21)
```

```
    stats: one Statistic,          // Calculated stats (avg speed, etc)
    weather: lone Weather_info,    // Weather at the time of the trip (R32)
    owner: one Registered_user,    // The user who recorded it
    path: one Path,                // The path this trip was on
    tripVisibility: one Visibility, // Visibility for stats aggregation (R26) (choice
        in save flow: R39)
    detections: set sensor_event   // Sensor events logged on this trip (R23)
}

// A raw sensor-detected event (e.g., from accelerometer)
// Its state evolves according to the sensor_event state diagram.
sig sensor_event {
    location: one GPS_point,       // Where the event happened
    trip: one Trip,                // The trip it happened on
    var state: one DetectionState // Logged -> AwaitingConfirmation ->
        {Confirmed, Ignored}
}

// Abstract signature for all user-submitted reports
abstract sig Report {
    timestamp: one Timestamp,      // When the report was made (freshness)
    author: one Registered_user, // Who made the report
    path: one Path                 // The path this report is about
}

// A report about the path's general condition
sig Status_report extends Report {
    status: one PathStatus
}

// A report about a specific obstacle
sig Obstacle_report extends Report {
    type: one ObstacleType,        // Type of obstacle
    location: one GPS_point,       // Location of obstacle
    // This association is mutable: it is created when an event is confirmed.
    var triggeredBy: lone sensor_event
}

// ----- 4. Static Invariants (Facts) -----
```

```
// Facts ensuring bidirectional relationships are consistent.
fact BidirectionalConsistency {
    trips = ~owner            // Trip.owner
    createdPaths = ~creator   // Path.creator
    submittedReports = ~author // Report.author
    reports = ~path           // Report.path
    performances = ~path      // Trip.path
    detections = ~trip        // sensor_event.trip
}

// A sensor_event can trigger at most one Obstacle_report.
fact OneReportPerEvent {
    all e: sensor_event | lone e.~triggeredBy
}

// Logic for sensor-confirmed obstacle reports
// (manual reports have no triggeredBy).
fact ObstacleSensorConsistency {
    all r: Obstacle_report | some r.triggeredBy implies (
        r.path = r.triggeredBy.trip.path and
        r.author = r.triggeredBy.trip.owner and
        r.triggeredBy in r.triggeredBy.trip.detections
    )
}

// Privacy rule: A 'Public' Trip must be on a 'Public' Path. (R25, R26, R39)
fact PublicTripOnPublicPath {
    all t: Trip |
        t.tripVisibility = Public implies t.path.pathVisibility = Public
}

// Privacy rule: A 'Private' Path can only have performances (Trips)
// recorded by the path's creator. (R25, R26, R39)
fact PrivatePathOwnedTrips {
    all p: Path |
        p.pathVisibility = Private implies
            all t: p.performances | t.owner = p.creator
}
```

```
// ------ 5. Dynamic Model for sensor_event & Obstacle_report ------
// Uses mutable fields (var) and Alloy 6 temporal operators.
// Models the state diagram from Figure 2.3 of the RASD:
// Logged -> AwaitingConfirmation -> Confirmed / Ignored.

// --- 5.1 Transition predicates ---

// The trip has ended and the event enters the review screen.
pred startReview[e: sensor_event] {
    e.state = Logged
    e.state' = AwaitingConfirmation
    // Frame condition: all other events keep their state.
    all e2: sensor_event - e | e2.state' = e2.state
    // No report is created/modified in this step.
    triggeredBy' = triggeredBy
}

// The user confirms a detection; an Obstacle_report is created/linked.
pred confirmDetection[e: sensor_event, r: Obstacle_report] {
    e.state = AwaitingConfirmation
    e.state' = Confirmed

    // The chosen report was not already associated.
    no r.triggeredBy
    r.triggeredBy' = e

    // Frame conditions
    all e2: sensor_event - e | e2.state' = e2.state
    all r2: Obstacle_report - r | r2.triggeredBy' = r2.triggeredBy
}

// The user rejects a detection.
pred rejectDetection[e: sensor_event] {
    e.state = AwaitingConfirmation
    e.state' = Ignored

    // Frame conditions
    all e2: sensor_event - e | e2.state' = e2.state
```

```
    triggeredBy' = triggeredBy
}

// No change in any mutable state.
pred doNothing {
    state' = state
    triggeredBy' = triggeredBy
}

// --- 5.2 Global behaviour & liveness ---

fact DetectionLifecycle {
    // Initial state: all events are Logged and no report is associated.
    all e: sensor_event | e.state = Logged
    no triggeredBy

    // At each step, one of the basic operations occurs.
    always (
        doNothing
        or some e: sensor_event | startReview[e]
        or some e: sensor_event, r: Obstacle_report | confirmDetection[e, r]
        or some e: sensor_event | rejectDetection[e]
    )
}

// Fairness: an event in AwaitingConfirmation will eventually
// be either confirmed or ignored (it cannot remain pending forever).
fact Fairness {
    always (all e: sensor_event |
        e.state = AwaitingConfirmation implies
            eventually (e.state = Confirmed or e.state = Ignored)
    )
}

// ----- 6. Static Model Verification (as in the RASD) -----

// PREDICATE to find a rich static instance.
pred FindPublicPathWithActivity {
    some p: Path {
```

```
        // A Public Path (R39)
        p.pathVisibility = Public

        // With at least two Trips on it, from different owners
        some disj t1, t2: Trip {
            t1.path = p and t2.path = p
            t1.tripVisibility = Public
            t2.tripVisibility = Private
            t1.owner != t2.owner
        }

        // And at least one obstacle report
        some r: Obstacle_report | r.path = p
    }
}

// COMMAND to run the predicate (purely static analysis)
run FindPublicPathWithActivity
    for 3 Path, 3 Trip, 3 Registered_user,
        5 GPS_point, 3 Statistic, 3 Weather_info,
        3 Timestamp, 3 Report, 3 sensor_event

// ASSERTION to check the Public Trip/Path privacy rule (R39)
assert PublicTripsOnPublicPaths {
    all t: Trip |
        t.tripVisibility = Public implies t.path.pathVisibility = Public
}

// ASSERTION to check the Private Path privacy rule (R39)
assert PrivatePathPrivacy {
    all p: Path |
        p.pathVisibility = Private implies
            all t: p.performances | t.owner = p.creator
}

// COMMANDS to check the assertions (static checks)
check PublicTripsOnPublicPaths for 5
check PrivatePathPrivacy for 5
```

```
// ----- 7. Dynamic Model Verification -----

// Safety: whenever an event is Confirmed,
// there exists an Obstacle_report linked to it.
assert ConfirmedImpliesReport {
    always (all e: sensor_event |
        e.state = Confirmed implies
            some r: Obstacle_report | r.triggeredBy = e
    )
}

// Liveness: every event that reaches AwaitingConfirmation
// is eventually decided (Confirmed or Ignored).
assert AwaitingEventuallyDecided {
    always (all e: sensor_event |
        e.state = AwaitingConfirmation implies
            eventually (e.state = Confirmed or e.state = Ignored)
    )
}

// COMMANDS to check the dynamic assertions
check ConfirmedImpliesReport for 3 but 10 steps
check AwaitingEventuallyDecided for 3 but 10 steps

// Example predicate to visualize a full lifecycle:
// an event that starts Logged and is eventually Confirmed.
pred showEventLifecycle {
    some e: sensor_event |
        e.state = Logged and
        eventually e.state = Confirmed
}

// Run the dynamic scenario for a bounded trace.
run showEventLifecycle for 3 but 5 steps
```

4.3. Results

```
Executing "Run FindPublicPathWithActivity for 3 Path, 3 Trip, 3 Registered_user, 5 GPS_point, 3 Statistic, 3 Weather_info, 3 Timestamp, 3 Report, 3 sensor_event"
  Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
  1..1 steps. 3548 vars. 291 primary vars. 6117 clauses. 629ms.
  Instance found. Predicate is consistent. 66ms.

Executing "Check PublicTripsOnPublicPaths for 5"
  Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
  1..10 steps. 116992 vars. 8621 primary vars. 221633 clauses. 1610ms.
  No counterexample found. Assertion may be valid. 15ms.

Executing "Check PrivatePathPrivacy for 5"
  Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
  1..10 steps. 231256 vars. 17001 primary vars. 439089 clauses. 1028ms.
  No counterexample found. Assertion may be valid. 26ms.

Executing "Check ConfirmedImpliesReport for 3 but 10 steps"
  Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=3 SkolemDepth=1 Symmetry=20 Mode=batch
  1..10 steps. 283168 vars. 20606 primary vars. 532291 clauses. 4537ms.
  No counterexample found. Assertion may be valid. 1192ms.

Executing "Check AwaitingEventuallyDecided for 3 but 10 steps"
  Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=3 SkolemDepth=1 Symmetry=20 Mode=batch
  1..10 steps. 335625 vars. 24211 primary vars. 629255 clauses. 848ms.
  No counterexample found. Assertion may be valid. 67ms.

Executing "Run showEventLifecycle for 3 but 5 steps"
  Solver=sat4j Steps=1..5 Bitwidth=4 MaxSeq=3 SkolemDepth=1 Symmetry=20 Mode=batch
  1..10 steps. 346461 vars. 25051 primary vars. 647929 clauses. 102ms.
  Instance found. Predicate is consistent. 39ms.
```

Figure 4.1: Formal analysis results.

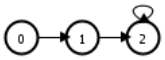

Figure 4.2: Sequence of states.

4.3.1. States 0

Figures 4.3 and 4.4 show State 0 of the execution trace produced by the run command used to visualize a complete `sensor_event` lifecycle. At this initial state, sensor events are still in the Logged phase: they represent raw detections collected during the recording session, before any user validation occurs. Consequently, no validated obstacle report is required at this point for the selected event, since the review process has not started yet.

This state is useful to highlight the separation between "data acquisition" and "data validation" in the model: the system may collect multiple detections while recording, but these detections remain tentative until the user explicitly reviews them. In addition, the instance already includes different visibility configurations (Public/Private) for paths and trips, allowing us to observe that privacy constraints are enforced structurally, independently from the later review process.

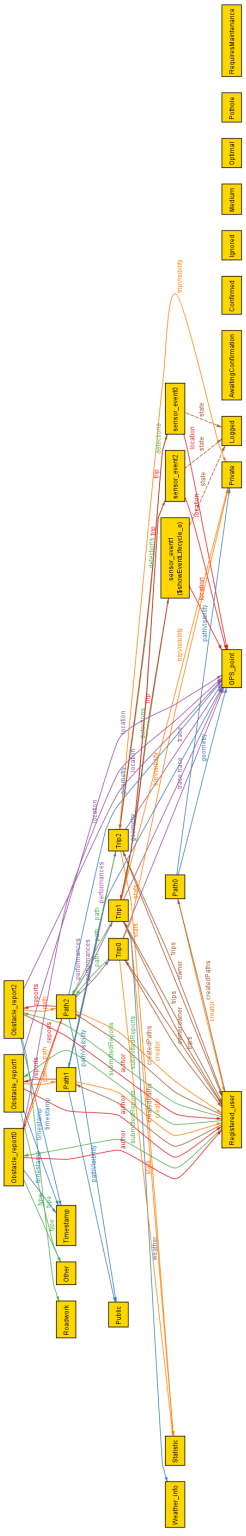

Figure 4.3: State 0 Relational Graph.

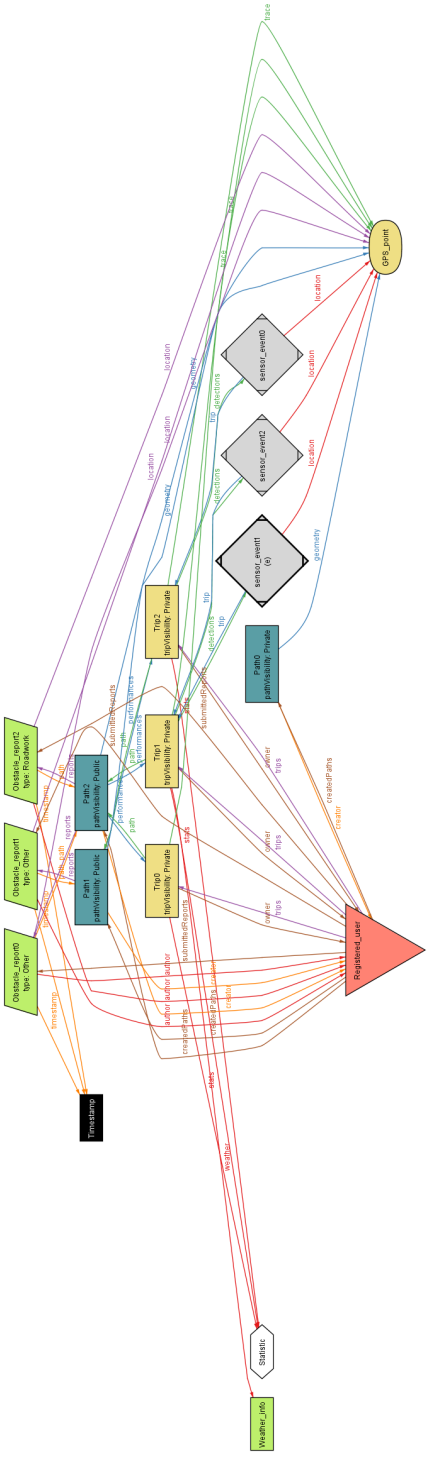

Figure 4.4: State 0 Projected View

4.3.2. States 1

Figures 4.5 and 4.6 show State 1, where the model enters the review phase. After the recording is stopped, the selected `sensor_event` moves from Logged to AwaitingConfirmation, reflecting the moment in which the application presents the user with pending detections to validate.

A key aspect to notice is that, even though the event is now under review, the model still does not associate it with a validated obstacle report: the transition to AwaitingConfirmation alone does not "publish" information. This explicitly captures the intended semantics that user interaction is needed to turn a raw detection into a confirmed report. Moreover, the rest of the domain structure (users, paths, trips, and other events) remains stable across the transition, which is consistent with the frame conditions used in the dynamic model.

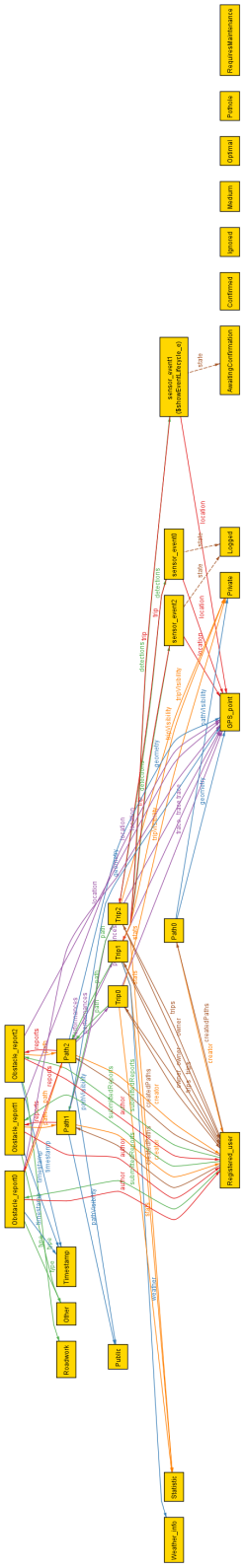

Figure 4.5: State 1 Relational Graph.

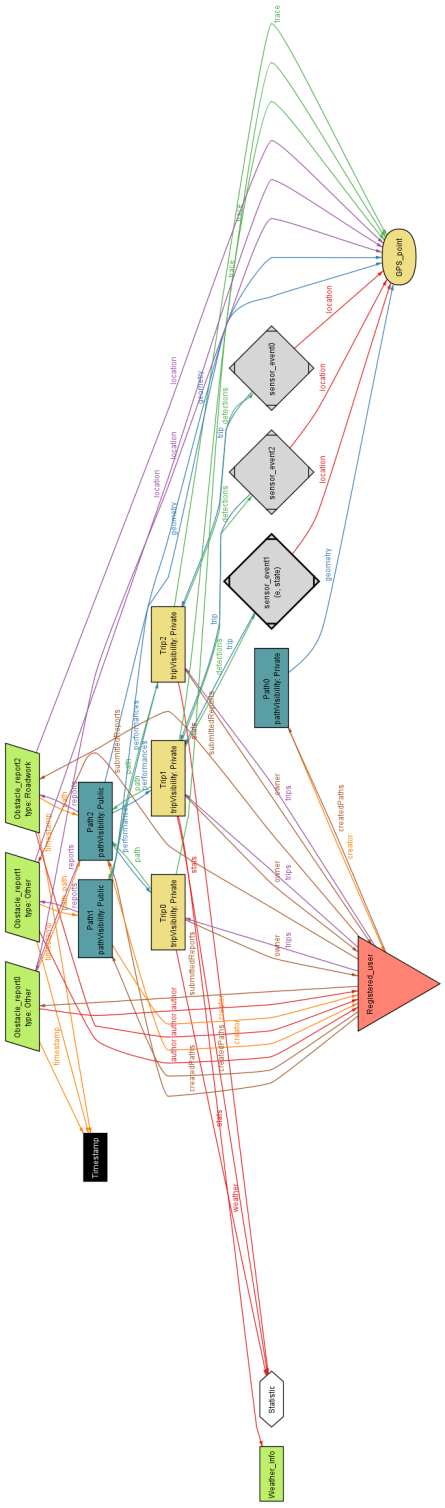

Figure 4.6: State 1 Projected View

4.3.3. States 2

Figures 4.7 and 4.8 show State 2, which completes the lifecycle for the selected event. In this state, the user has confirmed the detection: the `sensor_event` reaches the Confirmed state and the model introduces (or activates) the link to an `Obstacle_report` through the triggeredBy relation.

This state visually demonstrates the central safety property of the dynamic model: whenever an event is Confirmed, there must exist a corresponding obstacle report connected to it. In other words, only user-confirmed detections become official reports, while rejected detections never produce reports. The instance also illustrates the consistency constraints between the report and the trip/path context in which the event occurred, ensuring that validated information remains coherent with the recorded session.

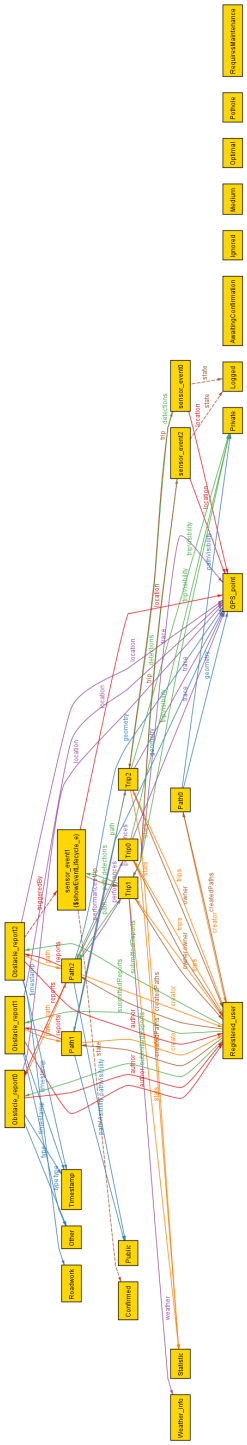

Figure 4.7: State 2 Relational Graph.

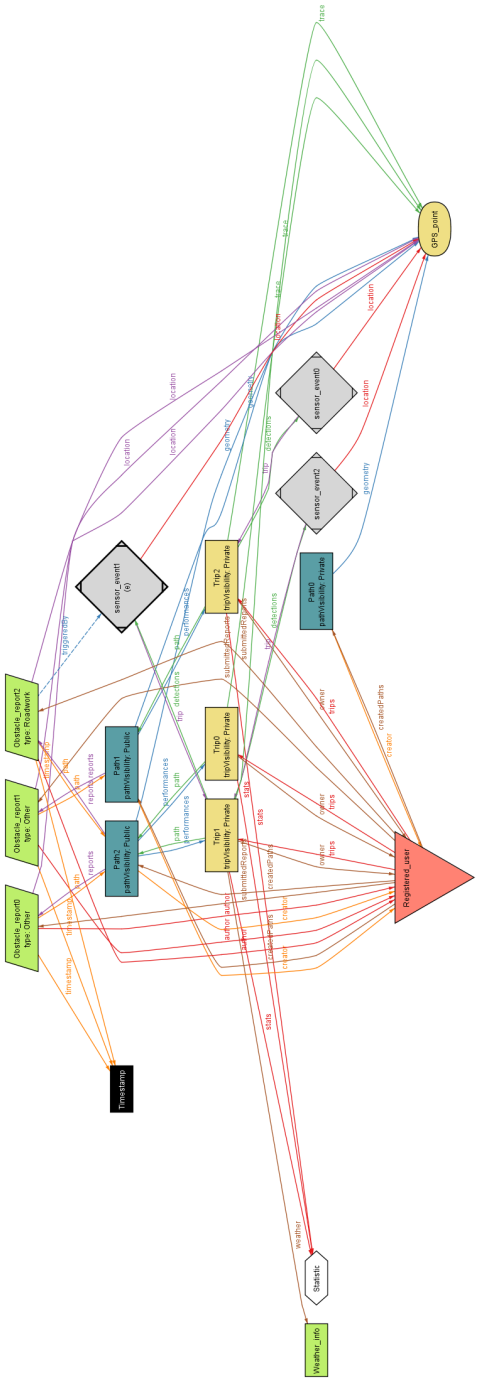

Figure 4.8: State 2 Projected View

5 | Effort Spent

Member of group	Effort spent	
Bellosi Jacopo	Introduction	5h
	Overall description	15h
	Specific requirements	20h
	Formal analysis	3h
	Reasoning	10h
Montanelli Matteo	Introduction	6h
	Overall description	10h
	Specific requirements	17h
	Formal analysis	3h
	Reasoning	12h
Tempobono Manuel	Introduction	4h
	Overall description	10h
	Specific requirements	15h
	Formal analysis	13h
	Reasoning	9h

Table 5.1: Effort spent by each member of the group.

6 References

6.1. References

- ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering.
- The specification document for the requirements engineering and design project AY 2025–2026.

6.2. Used Tools

- LaTeXand *Overleaf* as a text editor
- *Draw.io* for UML and state diagrams
- *Sequencediagram.org* for UML sequence diagrams

List of Figures

List of Tables